



河南大學
HENAN UNIVERSITY

第2章 运算方法和运算器

>>> 本章内容

- 数据的表示方法及其机器存储
- 算术运算和逻辑运算的运算方法（算法）
- 运算器的组成（逻辑实现）

2.1 数据与文字的表示方法 (掌握)

2.2 定点加法、减法运算 (掌握)

2.3 定点乘法运算 (理解)

2.4 定点除法运算 (理解)

2.5 定点运算器的组成 (了解)

2.6 浮点运算方法和浮点运算器 (理解)

>>> 2.1 数据与文字的表示方法

2.1.1 数据格式

2.1.2 数的机器码表示

2.1.3 字符与字符串的表示方法

2.1.4 汉字的表示方法

2.1.5 校验码

>>> 数据的类型（不同资料划分）

➤ 按数制分：

□ 十进制（D）、二进制（B）、十六进制（H）；

➤ 按数据格式分：

□ 真值：没有经过编码的数据表示方式；带正负号的数据，任何数制均可；

□ 机器数：符号化后的数值表示；位数固定，一般为字节整倍数；

➤ 按数据的表示范围分：

□ 定点数：小数点位置固定，数据表示范围小；

□ 浮点数：小数点位置不固定，数据表示范围较大。

➤ 按能否表示负数分：

□ 无符号数：数据所有位均为表示数值，只能表示正数；

□ 有符号数：有正负之分，最高位为符号位，其余位表示数值。

原码、反码、
补码、移码

>>> 数据的类型（本课程划分）

➤ 按数据类型分：

□ 数值数据

- ✓ 定点格式：数值范围有限，处理简单
- ✓ 浮点格式：数值范围很大，处理比较复杂

□ 符号数据

- ✓ ASCII码
- ✓ 汉字

➤ 选择数据的表示方式需要考虑的因素

□ 要表示的数的类型

□ 可能需要的数值范围

□ 数值精度

□ 数据存储和处理所需要的硬件代价

下列数值中最大的为（ ）。

A $(101001)_2$

B $(2000)_3$

C $(52)_7$

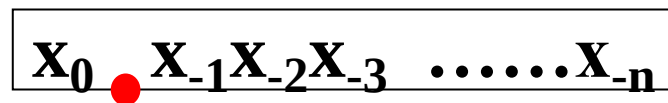
D $(2E)_{16}$

提交

>>> 2.1.1 数据格式——定点数（两种）

➤ **定点数：** 小数点固定在某一位置的数据； 设采用 $n+1$ 位数据

□ **纯小数：**



✓ **表示形式**

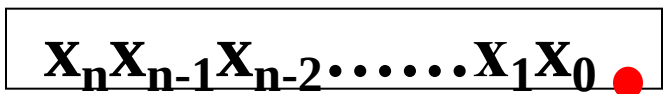
有符号数

$$x = x_s x_{-1} x_{-2} \dots x_{-n} \quad |x| \leq 1 - 2^{-n} \quad ; x_s \text{ 为符号位}$$

无符号数

$$x = x_0 x_{-1} x_{-2} \dots x_{-n} \quad 0 \leq x \leq 1 - 2^{-n} \quad ; x_0 = 0$$

□ **纯整数：**



✓ **表示形式**

有符号数

$$x = x_s x_{n-1} \dots x_1 x_0 \quad |x| \leq 2^n - 1 \quad ; x_s \text{ 为符号位}$$

无符号数

$$x = x_n x_{n-1} \dots x_1 x_0 \quad 0 \leq x \leq 2^{n+1} - 1 \quad ; x_n \text{ 为数值位}$$

注意

小数点位置是
机器约定好的，
一般不会保存。

>>> 定点数原码表示时最值

➤ 定点数：最值情况

设采用n+1位数据

□ 纯小数：（原码表示时最值情况）

✓ 表示形式 $2^0 \quad 2^{-1} 2^{-2} 2^{-3} \quad \dots \quad 2^{-(n-1)} \quad 2^{-n}$

✓ 最大正数

0	1	1	1	...	...	1	1
---	---	---	---	-----	-----	---	---

✓ 最大正数值为： $1-2^{-n}$,

✓ 最小正数

0	0	0	0	...	...	0	1
---	---	---	---	-----	-----	---	---

✓ 最小正数的值为： 2^{-n}

>>> 定点数原码表示时最值

➤ 定点数：最值情况

设采用n+1位数据

□ 纯小数：（原码表示时最值情况）

✓ 表示形式

✓ 绝对值最大负数

$$\begin{array}{ccccccc} 2^0 & 2^{-1} & 2^{-2} & 2^{-3} & & 2^{-(n-1)} & 2^{-n} \\ \hline 1 & 1 & 1 & 1 & \dots & 1 & 1 \end{array}$$

✓ 绝对值最大负数值为：- (1-2⁻ⁿ)

>>> 定点数补码表示时最值

➤ 定点数：最值情况

□ 纯小数：（补码表示时最值情况）

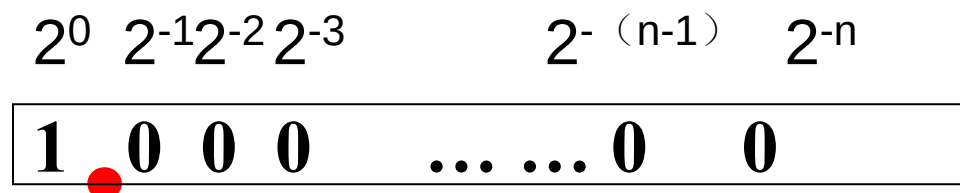
✓ 表示形式

✓ 绝对值最大负数

✓ 绝对值最大负数值为：-1

- 通过2.1节公式可推出此结论

□ 推导理解（在 $n+1$ 位情况下，定点小数用原码无法表示-1.0），见下页



>>> 定点数补码表示时最值

纯小数 绝对值最大的负数值 -1，推导理解（8位为例）

补码最小值为-1.0的得出逻辑：

先看倒数第二小的值是- $(1/2 + 1/4 + 1/8 + 1/16 + 1/32 + 1/64 + 1/128) = -0.9921875$

该值的原码：1.1111111

该值的反码：1.0000000

该值的补码：1.0000001

$$\begin{array}{r} \text{再看 } -1.0 \quad - (1/2 + 1/4 + 1/8 + 1/16 + 1/32 + 1/64 + 1/128) \\ \quad \quad \quad + \quad \quad \quad \quad \quad \quad \quad \quad \quad \quad \quad - (1/128) \\ \hline -1.0 \end{array}$$

所以，其实-1.0的二进制原码要占九位：11.0000000 依然硬气地假设最左位为符号位。

继续，所以-1.0的二进制反码也占九位：10.1111111

末尾加1得 -1.0的二进制补码也占九位：11.0000000

但是机器是8位机，所以数值位的最高位的1就隐去不要了，得到二进制 $1.0000000 = -1.0$ (十进制)

当机器中出现一个二进制补码1.0000000时，按照计算补码的逆规则可以得到其原码值：

1. 数值位在末尾减1，也就是减 $1/128$ 。一看是7个0，这时必须从高位借位1，这个1就是数值1.0。

减去 $1/128$ 后得到 1.1111111

2. 各数值位取反得到1.0000000。这个值看上去是-0.0，但是逆计算过程中借位时引入了1.0值，所以这个数是-1.0。



➤ 定点数：最值情况

□ 纯小数：（补码表示时最值情况）

✓ 绝对值最大负数

$2^0 \quad 2^{-1} \quad 2^{-2} \quad 2^{-3} \quad \dots \quad 2^{-(n-1)} \quad 2^{-n}$

1	0	0	0	...	...	0	0
---	---	---	---	-----	-----	---	---

✓ 绝对值最大负数值为：-1.0

- 通过2.2节公式可推出此结论（定点小数中，原码无法表示-1）

□ 纯小数补码最值-1.0其他推导方法

✓ 在 $n+1$ 位纯小数原码的表示方式中不存在-1.0，除非 $n+2$ 位能表示出：例如右图，所以 $n+1$ 位无法用原码推导出对应补码形式

✓ 但由于补码0的存在是唯一的，

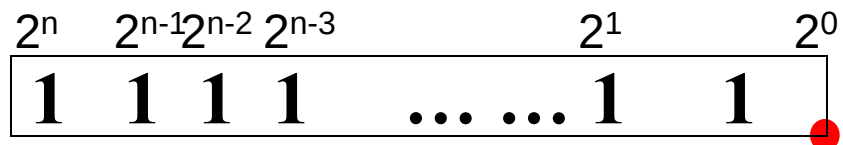
在指定的 $n+1$ 位数的情况下，多表达了-1.0这个数



➤ 定点数：最大值情况

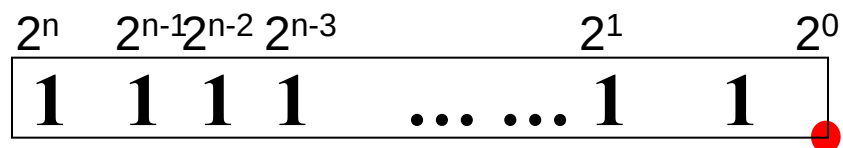
□ 纯整数：

✓ 原码表示的绝对值最大负数



- 原码表示的绝对值最大负数值为：- $(2^n - 1)$

✓ 补码表示的绝对值最大的负数



- 补码表示的绝对值最大的负数值为：- 2^n

>>> 2.1.2 数的机器码表示 (原码、补码、移码)

➤ **原码**：符号位 (0-正数, 1-负数) , 数值位与真值相同;

□ **n+1位数据的表示范围**

8位原码定点整数表示范围

$$-127 \leq X \leq +127$$

✓ 定点小数: $[-(1-2^{-n}), +(1-2^{-n})]$; 定点整数: $[-(2^n-1), +(2^n-1)]$

□ 0有两种表示法: $[+0]_{\text{原}} = 0000\ 0000$; $[-0]_{\text{原}} = 1000\ 0000$

□ 参与运算复杂, 需要将数值位与符号位分开考虑。

➤ **补码**：符号位 (0-正数, 1-负数) , 正数数值与真值相同, 负数数值各位取反末位加1;

□ **n+1位数据的表示范围**

✓ 定点小数: $[-1, +(1-2^{-n})]$; 定点整数: $[-2^n, +(2^n-1)]$

□ 0只有唯一的一种编码: $[+0]_{\text{补}} = [-0]_{\text{补}} = 0000\ 0000$

8位补码定点整数表示范围

$$-128 \leq X \leq +127$$

□ 只要结果不溢出, 可将补码符号位与数值位一起参与运算。

□ $[[x]_{\text{补}}]_{\text{补}} = [x]_{\text{原}}$

>>> 由原码求补码

➤ 由原码求补码的简便原则(负数)

方法一：

除符号位以外,其余各位按位取反, 末位加1;

方法二：

除符号位以外,从最低位开始,遇到的第一个1以前的各位保持不变, 之后各位取反。

例: $[X]_{\text{原}} = 1\ 1\ 0\ 1\ 1\ 0\ \mathbf{1}\ 0\ 0$

$[X]_{\text{补}} = 1\ 0\ 1\ 0\ 0\ 1\ \mathbf{1}\ 0\ 0$

>>> 求相反数的补码

➤ 由 $[X]_{\text{补}}$ 求 $[-X]_{\text{补}}$

□ 连同符号位的所有位一起取反，末位加1；

➤ 例： $X = -0.010\ 1011$

□ $[X]_{\text{补}} = 1.101\ 0101$; $[-X]_{\text{补}} = 0.010\ 1011$;

➤ 即： $[X]_{\text{补}} = 1\ 1\ 0\ 1\ 0\ 1\ 0\ 1$

$0\ 0\ 1\ 0\ 1\ 0\ 1\ 0$

1

+

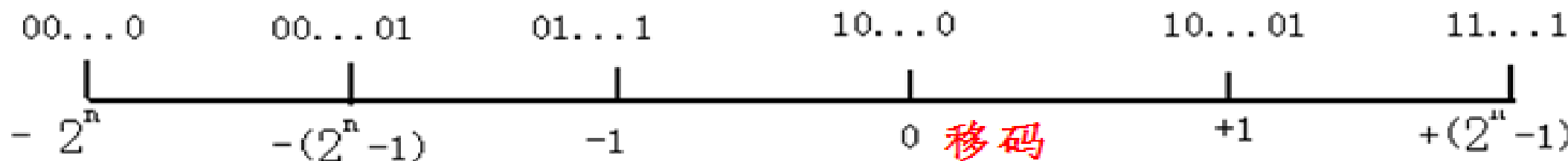
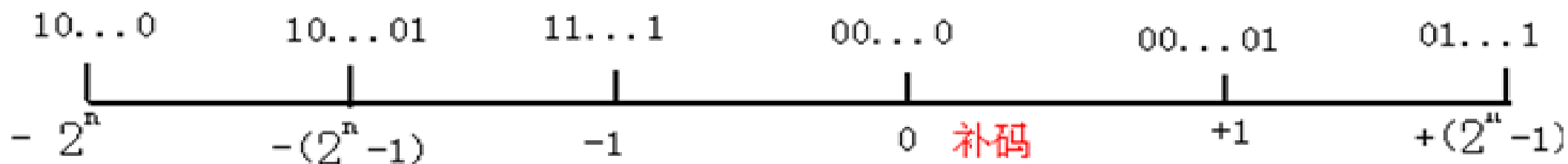
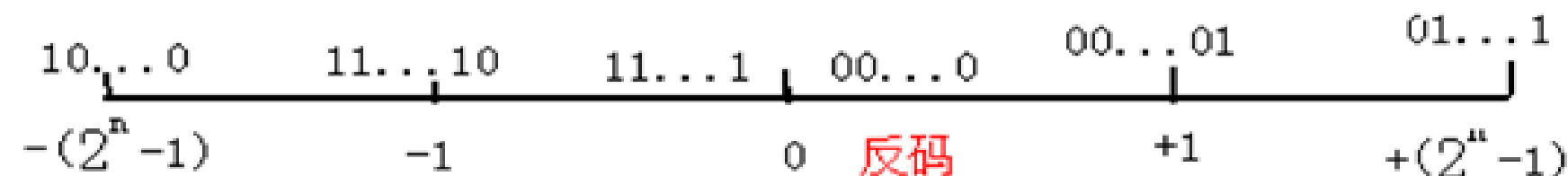
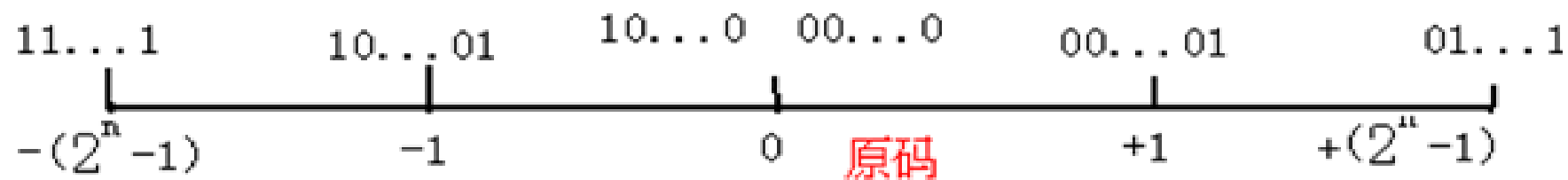
$[-X]_{\text{补}} = 0\ 0\ 1\ 0\ 1\ 0\ 1\ 1$

由 $[-X]_{\text{补}}$ 求 $[X]_{\text{补}}$ ，
此规则同样适用。

课本P22例6

以定点整数为例,用数轴形式说明原码、反码、补码、移码表示范围和可能的数码组合情况。

你能发现
什么规律



2015年考研真题第13题

13. 由3个“1”和5个“0”组成的8位二进制补码，能表示的最小整数是 ____。

A -126

B -125

C -32

D -3

提交

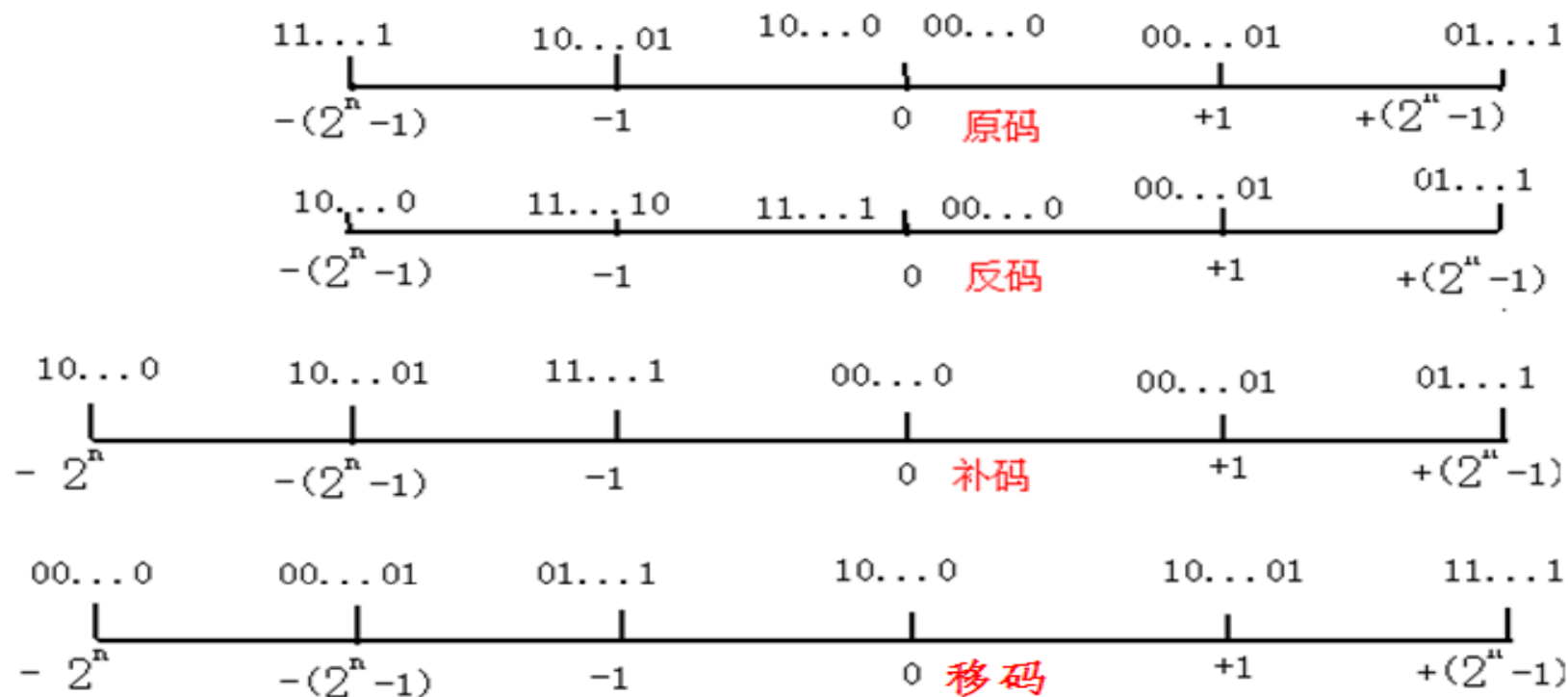
>>> 解答

➤ 3个“1”和5个“0”，符号位为“1”，然后将其他的“1”尽量放在低位，为了得到最小的整数，这样可以使负数的绝对值最大，从而得到最小的整数。

➤ 补码：10000011

原码：11111101

真值：-125



2016考研真题第13题

有如下C语言程序段

```
Short si=-32767;
```

```
Unsigned short usi=si;
```

执行上述两条语句后，usi的值为（ ）

short为16位有符号数；

unsigned short 为16位无符号数。

- ☐ A -32767
 ☐ B 32767
 ☐ C 32768
 ☒ D 32769

提交

➤ 有如下C语言程序段

```
Short si=-32767;
```

```
Unsigned short usi=si;
```

执行上述两条语句后，usi的值为（ ）

- short为16位有符号数，类型的范围是 -32768 ~32767;
- unsigned short 为16位无符号数，类型的范围0~65535 。
- si=- -32767 对应的补码是 1000000000000001。
- Unsigned short usi=si; 此时usi的二进制码被解释为无符号数 1000 0000 0000 0001 ， 因此即为32769

13. 已知带符号整数用补码表示，变量 x, y, z 的机器数分别为 FFFDH, FFDFH, 7FFCH，下列结论中，正确的是（ ）。

- A. 若 x, y 和 z 为无符号整数，则 $z < x < y$
- B. 若 x, y 和 z 为无符号整数，则 $x < y < z$
- C. 若 x, y 和 z 为带符号整数，则 $x < y < z$
- D. 若 x, y 和 z 为带符号整数，则 $y < x < z$

13. 32 位补码所能表示的整数范围是（ ）。

- A. $-2^{32} \sim 2^{31} - 1$
- B. $-2^{31} \sim 2^{31} - 1$
- C. $-2^{32} \sim 2^{32} - 1$
- D. $-2^{31} \sim 2^{32} - 1$

>>> 2023、2024考研题

13. 若 short 型变量 $x = -8190$ ，则 x 的机器数是 ()。

- A. E002H B. E001H C. 9FFFH D. 9FFE H

12. C语言代码段如下，执行该代码段后， j 的值是 ()。

```
int i=32777;  
short si=i;  
int j=si;
```

- A. -32777 B. -32759 C. 32759 D. 32777

408历年真题解析 <https://zhuanlan.zhihu.com/p/3484668199>

>>> 移码表示法

你能总结出移码的哪些规律？

➤ 移码：定点整数，一般用于表示浮点数的阶码；

➤ 定义：n+1位的移码表示为 $x_n x_{n-1} \dots x_1 x_0$

$$[x]_{\text{移}} = 2^n + x \quad -2^n \leq x < 2^n$$

由真值加一个固定的常数生成，这个固定的常数称为偏移量

□ 一般称为“移 2^n 码”；

✓ 如8位标准移码，称为移128码；

➤ 与补码关系：符号位相反，数值位相同；

➤ 优点：

□ 可以比较直观地判断两个数据的大小；

□ 表示浮点数阶码时，容易判断是否下溢；

✓ 当阶码为全0时，浮点数下溢。

4位补码与移码

真值	补码	移码
-8	1000	0000
-7	1001	0001
-6	1010	0010
.....		
0	0000	1000
+1	0001	1001
.....		
+7	0111	1111

课本P29例2.8

将十进制真值(- 127, - 1, 0, + 1, + 127)列表表示成二进制数及原码、反码、补码、移码值。(移码8位表示, 偏移量为128)

十进制真值	二进制真值	原码表示	反码表示	补码表示	移码表示
-127	-111 1111	1111 1111	1000 0000	1000 0001	0000 0001
-1	-000 0001	1000 0001	1111 1110	1111 1111	0111 1111
0	+000 0000	0000 0000	0000 0000	0000 0000	1000 0000
	-000 0000	1000 0000	1111 1111		
+1	+000 0001	0000 0001	0000 0001	0000 0001	1000 0001
+127	+111 1111	0111 1111	0111 1111	0111 1111	1111 1111

>>> 原码、补码、移码

➤ 正数:

- 原、补码的编码完全相同;
- 补码和移码的符号位相反, 数值位相同;

➤ 负数:

- 原码: 符号位为1
数值部分与真值的绝对值相同
- 补码: 符号位为1
数值部分与原码各位相反, 且末位加1
- 移码: 符号位与补码相反, 数值位与补码相同

➤ 注意: 有些最值的求法不能用上述编码转换规则

>>> 课本P30例2.9

设机器字长16位，定点表示，数值15位，数符1位，问：

(1) 定点原码整数表示时，最大正数是多少？最小负数是多少？

(2) 定点原码小数表示时，最大正数是多少？最小负数是多少？

➤ 定点原码整数

□ 最大正数

0	111 1111 1111 1111
---	--------------------

$$(2^{15}-1) = +32767$$

□ 最小负数

1	111 1111 1111 1111
---	--------------------

$$-(2^{15}-1) = -32767$$

若用补码表示呢？
最小正数、最大
负数各是多少呢？

➤ 定点原码小数

□ 最大正数

0	111 1111 1111 1111
---	--------------------

$$(1-2^{-15}) = +(1-1/32768)$$

□ 最小负数

1	111 1111 1111 1111
---	--------------------

$$-(1-2^{-15}) = -(1-1/32768)$$

>>> 定点数表示方法

- **优点：**表示方法简单，便于运算
- **缺点：**表示的数的范围有限，要表示很大或很小的数必须用很多比特；s
数据精度低等
- **大数量级数据的表示**
 - 定点计算机间接表示：在运算之前先按照一定的固定比例（比例因子）缩放，在运算之后再恢复
 - 用幂的方式运算
 - 引入浮点数

>>> 科学计数法的表示

➤ 一个十进制数可以表示成不同的形式：

$$(N)_{10} = 123.456 = 123456 \times 10^{-3} = 0.123456 \times 10^{+3}$$

➤ 同理，一个二进制数也可以有多种表示：

$$(N)_2 = 1101.0011 = 11010011 \times 2^{-100} = 0.1101\ 0011 \times 2^{+100}$$

2^{-100} 是比例因子，存储时将有效数字和比例因子分别存储，即分别存储数的表示范围与精度，比例因子变化即数的小数点变化，因此称为浮点表示法。

>>> 2.1.1 数据格式——浮点数

课本P23

➤ 机器数的一般表示形式

$$N = (-1)^S \times R^e \times M$$

数符S	阶符	阶码e	尾数M
阶符	阶码e	数符S	尾数M

- 基数R：计算机内部数据默认R=2，不需要占用数据位表示；
- 数符S：1位，表示浮点数的正负；
- 尾数M：有符号定点纯小数，默认小数点在尾数之前；表示数据的全部有效数位，决定着数值的精度；
- 阶码e：比例因子的指数，又称浮点数的指数，用于指出小数点在该数中的位置，决定着数据数值的大小。有符号定点纯整数，包含符号位（阶符）和数值位；
- 一个浮点数就可以有两个定点数表示
 - ✓ M称为浮点数的尾数，用一个定点小数来表示；
 - ✓ E称为浮点数的指数或阶码，用一个定点整数来表示。

>>> 浮点数举例

➤ 请将数据-12.625D用16位的浮点数形式表示。

□ 数据格式如下，阶码（含阶符）4位，尾数（含数符）12位；

阶符	阶码	数符	尾数
1位	3位	1位	11位

➤ $-12.625D = -1100.101 B = -0.1100 101 \times 2^4 B$

➤ 设尾数用原码表示，阶码用补码表示；

□ 浮点数表示为： 0 100 ; 1.110 0101 0000

➤ 设尾数用补码表示，阶码用移码表示；

□ 浮点数表示为： 1 100 ; 1.001 1011 0000

>>> 浮点数规格化

➤ 浮点数的一般表示

□ 同一个浮点数的表示是不唯一的。如 0.1101 可表示为 0.01101×2^1 ，也可表示为 1.101×2^{-1} 。

□ 机器数的表示不同，不利于运算。

➤ 规格化：是浮点数唯一形式的表示；

➤ 规格化的目的

□ 保证浮点数表示的唯一性；

□ 保留更多地有效数字，提高运算的精度。

>>> 浮点数规格化

➤规格化要求：当尾数的值不为0时， **$|\text{尾数}| \geq 0.5$** ；

□尾数**原码**表示：最高数值位为1，正数0.1...，负数1.1...；

✓例如， 0.011×2^5 要规格化则变为 0.11×2^4 ；

- 0.011×2^5 要规格化则变为 1.11×2^4 ；

高位上的0代表实际
绝对值对应位上的1

□尾数**补码**表示：最高数值位和符号位相反，正数0.1...，负数1.0...；

✓ $[+0.1B]_{\text{补}} = 0.10 \cdots 0$ $[-0.1B]_{\text{补}} = 1.10 \cdots 0$ $[-0.10 \cdots 01B]_{\text{补}} = 1.01 \cdots 11$

➤规格化处理：

□尾数向左移n位（小数点右移），同时阶码减n；——→ 左规

□尾数向右移n位（小数点左移），同时阶码加n。——→ 右规

>>> 浮点数规格化-----分析 (负数补码)

➤规格化要求：当尾数的值不为0时，**|尾数| ≥ 0.5**；

□尾数**补码**表示：最高数值位和符号位相反，正数0.1..., 负数1.0...；

➤按照规格化要求和尾数为纯小数两个条件，负数M范围应该是：**-1 < M ≤ - $\frac{1}{2}$**

□由上可知M补码表示，按照补码定义

定义：	$[x]_{\text{补}} = \begin{cases} x & 1 > x \geq 0 \\ 2+x = 2 - x & 0 \geq x \geq -1 \end{cases} \quad (\text{mod } 2)$
○定点小数：	

$$-1+2 < M+2 \leq 2+(-\frac{1}{2}) \quad \longrightarrow \quad 1 < M+2 \leq 1+\frac{1}{2} \quad \longrightarrow \quad 1 < [M]_{\text{补}} \leq 1+\frac{1}{2}$$

>>> 浮点数规格化-----分析（负数补码）规格化通式

➤按照规格化要求和尾数为纯小数两个条件，负数M范围应该为： $-1 < M \leq -\frac{1}{2}$

□M补码表示，按照定义 $\longrightarrow 1 < [M]_{\text{补}} \leq 1\frac{1}{2}$

真值	二进制表示
-1/2:	1.100*****00
	1.011*****11
	1.011*****10
	
	1.000*****01
-1:	1.000*****00

-1原本不在补码表示范围，但 $-\frac{1}{2}$ 与其他的数据1.0*****表示又不同，为了统一表示，计算机判断方便（例如异或电路实现符号判断），M补码表示为（去 -1/2，添加-1）：



尾数（负数）M规格化的表示应为：1.0*****

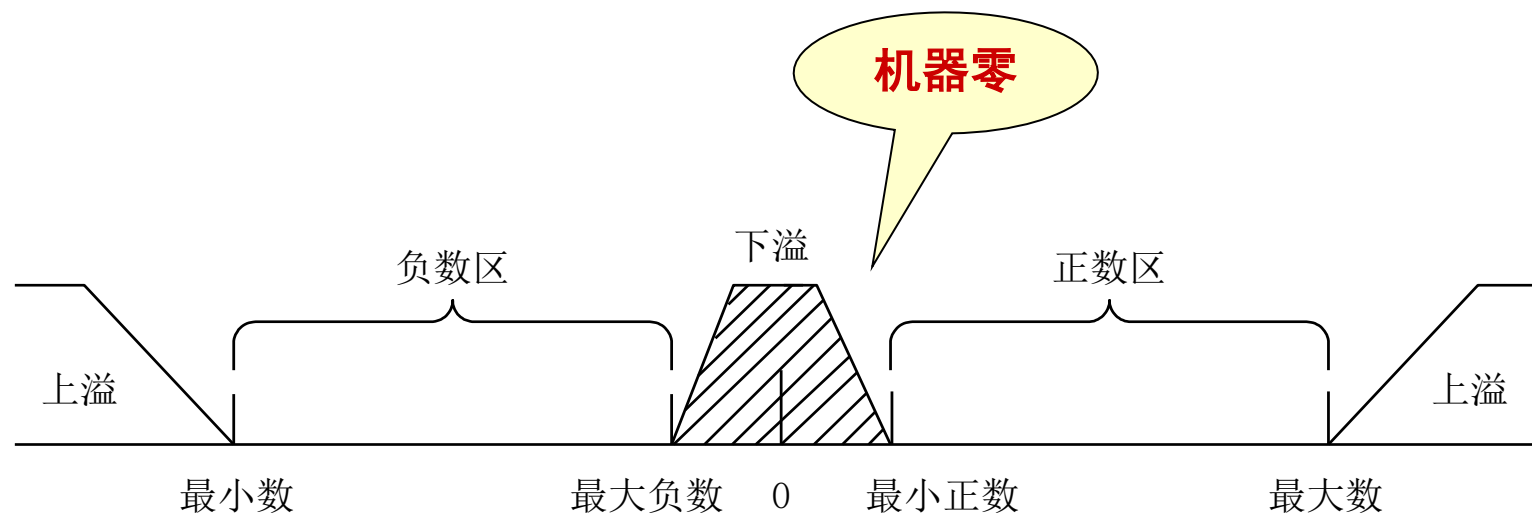
即 $M < 0$ ， $[-1]_{\text{补}} \leq [M]_{\text{补}} < [-1/2]_{\text{补}}$

>>> 浮点数的数据表示范围----机器零

➤ 在浮点数的表示范围中，有两种情况被称为机器零：

- (1) 若浮点数的尾数为零，无论阶码为何值；
- (2) 当阶码的值遇到比它能表示的最小值还要小时，无论其尾数为何值

➤ 机器零：浮点数的尾数为零或阶码小于机器能表示的最小阶码



>>> 浮点数的溢出

➤ 浮点数的溢出：数的大小超出了浮点数的表示范围

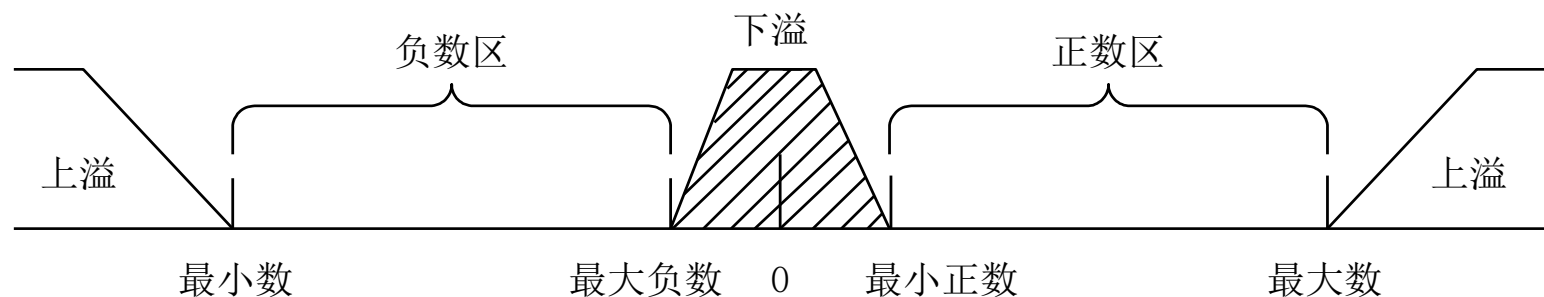
□ 原因：指数部分太大，无法用有限的指数字段表示出来

✓ 下溢：阶码小于机器能表示的最小阶码—— 一般作为机器零处理

✓ 上溢：阶码大于机器能表示的最大阶码—— 必须转为中断处理

➤ 如何减少溢出

□ 双精度格式



>>> 浮点数的数据表示范围



➤ 浮点数的溢出：阶码溢出

□ 上溢：阶码大于所能表示的最大值；

□ 下溢：阶码小于所能表示的最小值；

➤ 机器零：尾数为 0，或阶码小于所能表示的最小值；

浮点数的最值

移码定点整数范围
 $[-2^m, +(2^m-1)]$

补码定点小数范围
正数区 $[+2^{-n}, +(1-2^{-n})]$
负数区 $[-1, -2^{-n}]$

最大负数原码:
1.0000...01
补码: **1.1111...11**
规格化: 原码:
1.1000...01
对应补码: **1 0111...11**

设浮点数格式为



	非规格化数据		规格化数据	
	真值	机器数	机器数	真值
最小负数	$-1 \times 2^{+(2^m-1)}$	1 1...11; 1 00...00	同左	同左
最大负数	$-2^{-n} \times 2^{-2^m}$	0 0...00; 1 11...11	0 0...00; 1 01...11	$-(2^{-1}+2^{-n}) \times 2^{-2^m}$
最小正数	$+2^{-n} \times 2^{-2^m}$	0 0...00; 0 00...01	0 0...00; 0 10...00	$+2^{-1} \times 2^{-2^m}$
最大正数	$+(1-2^{-n}) \times 2^{+(2^m-1)}$	1 1...11; 0 11...11	同左	同左

>>> 定点数与浮点数的数据表示范围和精度比较

➤ 定点数

以16位数据为例，
补码表示

□ 定点整数： $-2^{15} \sim +2^{15}-1$ (即 $-32768 \sim +32767$)

□ 定点小数： $-1 \sim +1-2^{-15}$ (即 $-1 \sim +0.999\ 9694\ 8242\ 1875$)

➤ 浮点数

1位阶符	7位阶码	1位数符	7位尾数
------	------	------	------

□ 格式：

□ 最大正数： $+127 \times 2^{+127}B$

□ 最小正数： $+0.0000001 \times 2^{-127}B$

□ 最大负数： $-0.0000001 \times 2^{-127}B$

□ 最小负数： $-127 \times 2^{+127}B$

>>> 务必牢记

- 浮点数的表示要**与具体的格式规定**有关；
- 做题时，要看清题目里要求的浮点数各部分的**位数、排列和编码形式**；
- 一般机器中，浮点数采用IEEE754标准来存放float、double类型的变量；
 - IEEE754标准只是浮点数的一种表示形式；

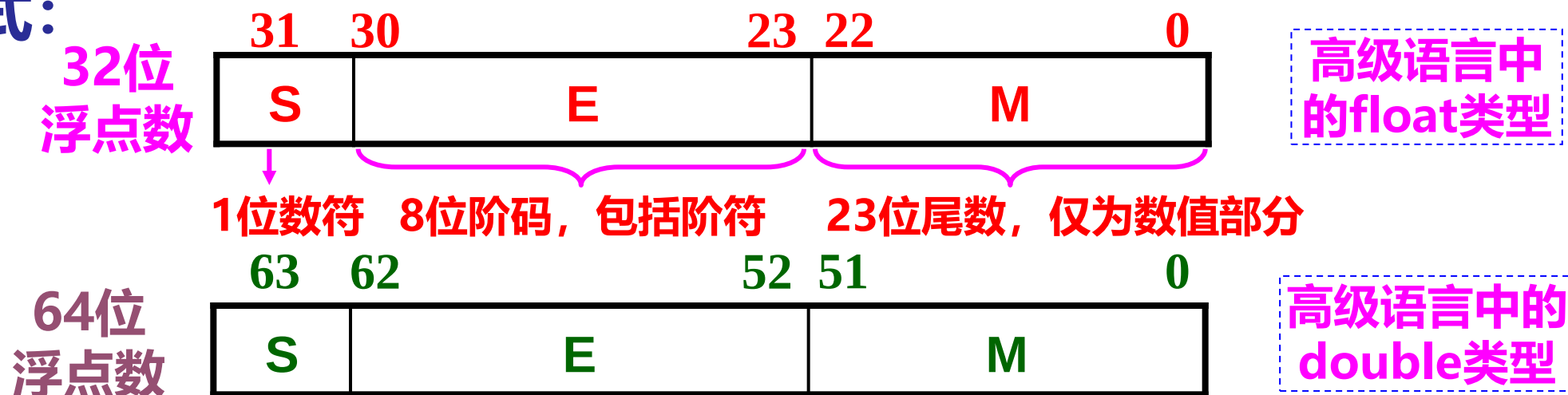
>>> 浮点数的IEEE754标准表示

➤ IEEE (Institute of Electrical and Electronics Engineers)

□ 美国电气及电子工程师学会

□ IEEE致力于电气、电子、计算机工程和与科学有关的领域的开发和研究，也是计算机网络标准的主要制定者。

➤ 为便于软件移植，IEEE754 标准规定了计算机内部32位/64位浮点数的标准格式：



>>> 32位浮点数的IEEE754 标准表示及其规定

数符S	阶码E (8位移码)	尾数M (23位原码)
-----	------------	-------------

- 数符S：表示浮点数的符号，占1位，0—正数、1—负数；
- 尾数M：23位，原码纯小数表示，小数点在尾数域的最前面；
 - 由于原码表示的规格化浮点数要求，最高数值位始终为1，因此该标准中隐藏最高数值位(1)，尾数的实际值为1.M；
- 阶码E：8位，采用有偏移值的移码表示；
 - 移127码，即 $E = e + (2^7 - 1) = e + 127$ ；
 - ✓ 标准8位移码应该是移128码， $[x]_{\text{移}} = x + 2^7 = x + 128$
- 浮点数的真值： $N = (-1)^S \times (1.M) \times 2^{E-127}$
- 指数（阶码）的最大值和最小值作为特殊预留，用来表示异常事件或机器零。
- 实际精度24位（加上隐藏位）

e 的取值范围为-127 ~ +128

>>> IEEE754 标准格式 (64位格式)

64位浮点数格式 (double类型)

数符S	阶码E (11位移码)	尾数M (52位原码)
-----	-------------	-------------

其真值表示为:

$$N = (-1)^S \times (1.M) \times 2^{E-1023} \quad e = E - 1023$$



754标准: $[X]_{\text{移}} = 2^{m-1} + X$, 127

最后将移码看做无符号数
注意只是看做而已

真值	原码		补码	移码	无符号数
-128			1000 0000	1111 1111	255
-127	1111 1111		1000 0001	0000 0000	0
-126	1111 1110		1000 0010	0000 0001	1
...	...		...	...	...
0	1000 0000		0000 000	0111 1111	127
1	0000 0001		0000 0001	1000 0000	128
...	...		...	...	...
126	0111 1110		0111 1110	1111 1101	253
+127	0111 1111		0111 1111	1111 1110	254

这两个特殊

>>> IEEE754 标准的数据表示

➤ IEEE754 标准中的阶码E 0000 0000 ~ 1111 1111

□ 正零、负零 E=0000 0000(真值-127), M=0000 ... 0000

✓ E与M均为零, 真值 $N=(-1)^s \times 0$, 机器0, 正负之分由数据符号确定;

□ 正无穷、负无穷 E=1111 1111 (真值-128), M=0000 ... 0000

✓ E为全1, M为全零, 真值 $N=(-1)^s \times \infty$, 正负之分由数据符号确定;

□ 阶码E的其余值 (0000 0001~1111 1110) 为规格化数据;

✓ E为1 ~ 254, 真正的指数e的范围为-126 ~ +127 ($e=E-127$)

➤ 为避免浮点数下溢, 尾数非0时, 阶码为0时, 允许采用比最小规格化数还小的非规格化数来表示, 但此时尾数M前的隐含位为0, 而不是1。

✓ 真值 $N=(-1)^s \times (0.M) \times 2^{-126}$ (非规格化)

>>> IEEE754 标准的数据表示

➤ IEEE754 标准中的阶码E表示范围：0000 0000 ~ 1111 1111

□ 当E=0000 0000时，阶码最小，浮点数N逼近下溢区；

✓ 同时M为全零时，浮点数N置为机器零，分正零和负零；

✓ 同时M不全为零时，为避免浮点数N进入下溢区，M前隐含位为0；

$$N = (-1)^S \times (0.M) \times 2^{-126}$$

□ 当E=1111 1111时，阶码最大，浮点数N逼近上溢区；

✓ 同时M为全零时，浮点数N置为正/负无穷；

✓ 同时M不全为零时，浮点数置为NaN（无定义数据）

□ 当E为0000 0001~1111 1110时，采用规格化数据表示真值；

$$N = (-1)^S \times (1.M) \times 2^{E-127}$$

有效阶码范围为-126~+127

>>> 课本P23 例2.5

例1. 若浮点数 x 的754标准存储格式为 $(41360000)_{16}$, 求其浮点数的十进制数

解:

$$\square (41360000)_{16} = \underbrace{0}_{\text{数符S}} \underbrace{100\ 0001\ 0}_{\text{阶码E}} \underbrace{0111\ 0110\ 0000\ 0000\ 0000\ 0000}_{\text{尾数M}}$$

$$\square \text{指数 } e = E - 127 = 1000\ 0010 - 0111\ 1111 = 0000\ 0011 = 3$$

$$\square \text{尾数 } 1.M = 1.0111\ 0110\ 0000\ 0000\ 0000\ 0000 = 1.011011$$

$$\begin{aligned}\square \text{浮点数 } N &= (-1)^S \times (1.M) \times 2^e \\ &= (-1)^0 \times (1.011011) \times 2^3 \\ &= (11.375)_{10}\end{aligned}$$

>>> 课本P29 例2.6

例2. 将 $(20.59375)_{10}$ 转换成754标准的32位浮点数的二进制存储格式。

解：

□ $(20.59375)_{10} = (10100.10011)_2$

□ 将尾数规范为1.M的形式：

$$10100.10011 = 1.010010011 \times 2^4 \quad e = 4$$

□ 可得：M = 010010011

$$S = 0$$

$$E = 4 + 127 = 131 = 1000\ 0011$$

□ 故，32位浮点数的754标准格式为：

$$0100\ 0001\ 1010\ 0100\ 1100\ 0000\ 0000\ 0000 = (41A4C000)_{16}$$

>>> 练习

将下列十进制数表示成IEEE754格式的32位浮点数形式存储。

(1) 27/32

(2) 11/512

解:

$$(1) \ 27/32 = 27 * (1/32) = (0001 \ 1011)_2 * 2^{-5}$$

✓ 尾数: 1.1011; 阶码: $e = -5 + 4 = -1$, $E = e + 127 = 126$

✓ IEEE754数据

0 0111 1110 1011 0000 0000 0000 0000 000

$$(2) \ 11/512 = (0000 \ 1011)_2 * 2^{-9}$$

✓ 尾数: 1.011 ; 阶码: $e = -9 + 3 = -6$, $E = e + 127 = 121$

✓ IEEE754数据

0 0111 1001 0110 0000 0000 0000 0000 00

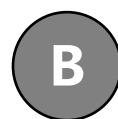
2011年考研统考题目 第13题

float型数据通常用IEEE754单精度浮点数格式表示。

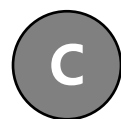
若编译器将float型变量x分配到一个32位浮点寄存器FR1中，且 $x = -8.25$ ，则FR1的内容是（ ）。



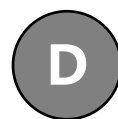
A C104 0000H



B C242 0000H



C C184 0000H



D C1C2 0000H

提交

>>> 2011年考研统考题目 第13题

➤ float型数据通常用IEEE754单精度浮点数格式表示。若编译器将float型变量x分配到一个32位浮点寄存器FR1中，且 $x = -8.25$ ，则FR1的内容是（ ）。

A. C104 0000H

B. C242 0000H

C. C184 0000H

D. C1C2 0000H

解答：

$$X = -8.25 = -1000.01 = -1.00001 \times 2^3$$

$$\therefore S=1 \quad M=000010...0 \quad E=127+3=130$$

数符S	阶码E (8位移码)	尾数M (23位原码)
1	100 0001 0	000 0100 0000 0000 0000 0000

2012年考研统考题目 第14题

12. float类型（即IEEE 754单精度浮点数格式）能表示的最大正整数是（ ）

A $2^{126}-2^{103}$

B $2^{127}-2^{104}$

C $2^{127}-2^{103}$

D $2^{128}-2^{104}$

提交

>>> 2012年考研统考题目 第14题

➤ 14.float类型（即IEEE 754单精度浮点数格式）能表示的最大正整数是（ ）

A. $2^{126}-2^{103}$

B. $2^{127}-2^{104}$

C. $2^{127}-2^{103}$

D. $2^{128}-2^{104}$

解答：

最大正整数： $1.M \times 2^{E-127}$ E的有效最大值为254

$$1.M \times 2^{E-127} = (1 + 1 - 2^{-23}) \times 2^{254-127}$$

$$= 2 \times 2^{127} - 2^{-23+127}$$

$$= 2^{128} - 2^{104}$$

2013年考研统考题目 第13题

13. 某数采用IEEE 754单精度浮点数格式表示为C640 0000H，则该数的真值为（ ）

☒ A -1.5×2^{13}

☐ B -1.5×2^{12}

☐ C -0.5×2^{13}

☐ D -0.5×2^{12}

提交

>>> 2013年考研统考题目 第13题

13. 某数采用IEEE 754单精度浮点数格式表示为C640 0000H, 则该数的真值为 () **A**

A. -1.5×2^{13}

B. -1.5×2^{12}

C. -0.5×2^{13}

D. -0.5×2^{12}

解答:

➤ 机器数: **1100 0110 0100 0000 0000**

➤ **$S=1$, $E=10001100$, $M=1.1$**

➤ **$N = (-1)^1 \times 1.1 + 1101 = -1.5 \times 2^{13}$**

>>> 2019、2020年考研题

13. 考虑以下 C 语言代码：

```
unsigned short usi = 65535;  
short si = usi;
```

执行上述程序段后，si 的值是 ()。

- A. -1 B. -32767 C. -32768 D. -65535

13. 已知带符号整数用补码表示，float 型数据用 IEEE 754 标准表示，假定变量 x 的类型只可能是 int 或 float，当 x 的机器数为 C800 0000H 时， x 的值可能是 ()。

- A. -7×2^{27} B. -2^{16} C. 2^{17} D. 25×2^{27}

>>> 2021、2022年考研题

13. 已知带符号整数用补码表示, 变量 x, y, z 的机器数分别为 FFFDH, FFDFH, 7FFCH, 下列结论中, 正确的是 ()。

- A. 若 x, y 和 z 为无符号整数, 则 $z < x < y$
- B. 若 x, y 和 z 为无符号整数, 则 $x < y < z$
- C. 若 x, y 和 z 为带符号整数, 则 $x < y < z$
- D. 若 x, y 和 z 为带符号整数, 则 $y < x < z$

14. 下列数值中, 不能用 IEEE 754 浮点格式精确表示的是 ()。

- A. 1.2
- B. 1.25
- C. 2.0
- D. 2.5

13. 32 位补码所能表示的整数范围是 ()。

- A. $-2^{32} \sim 2^{31} - 1$
- B. $-2^{31} \sim 2^{31} - 1$
- C. $-2^{32} \sim 2^{32} - 1$
- D. $-2^{31} \sim 2^{32} - 1$

14. -0.4375 的 IEEE 754 单精度浮点数表示为 ()。

- A. BEE0 0000H
- B. BF60 0000H
- C. BF70 0000H
- D. C0E0 0000H

>>> 2023、2024、2025年考研题

14. 已知 float 型变量用 IEEE 754 单精度浮点数格式表示。若 float 型变量 x 的机器数为 8020 0000H, 则 x 的值是 ()。

- A. -2^{-128} B. -1.01×2^{-127} C. -1.01×2^{-126} D. 非数 (NaN)

14. 某科学实验中, 需要使用大量的整型参数, 为了保证表数精度的基础上提高运算速度, 需要选择合理的数据表示方法。若整型参数 α 、 β 的取值范围分别为 $-2^{20} \sim 2^{20}$ 、 $-2^{40} \sim 2^{40}$, 则下列选项中, α 、 β 最适宜采用的数据表示方法分别是 ()。

- A. 32位整数、32位整数 B. 单精度浮点数、单精度浮点数
C. 32位整数、双精度浮点数 D. 单精度浮点数、双精度浮点数

13. 在 IEEE754 标准下, 4730 0000H 对应的真值是多少 ()。

- A. 0.375×2^{14} B. 1.6375×2^{14}
C. 0.375×2^{15} D. 1.375×2^{15}

【参考答案】D

12. 有一段代码:

```
Short si=-32767;
```

```
Unsigned int ui=si;
```

则其对应的值为: ()。

A. $2^{15}+1$ B. 2^{15} C. $2^{32}-2^{15}-1$ D. $2^{32}-2^{15}+1$

【参考答案】 D

13.在 IEEE754 标准下, 4730 0000H 对应的真值是多少 ()。

A. 0.375×2^{14} B. 1.6375×2^{14}

C. 0.375×2^{15} D. 1.375×2^{15}

【参考答案】 D

14. $x=A3H, y=75H$ 计算 $x-y$ 求真值和 OF ()。 D. 46,1

15. 结构体小端存储

```
{  
    int d;  
    char C [10];  
    int s;  
} M[200]
```

其起始地址为 0000 A0B0H 则 M[1].d 的机器数为 ()

- A. 0000 A0C3H B. 0000 A0C4H
C. 0000 A0C5H D. 0000 A0C6H



课本P30例2.10
(类似IEEE754标准的公式，但阶码采用的是移128码)

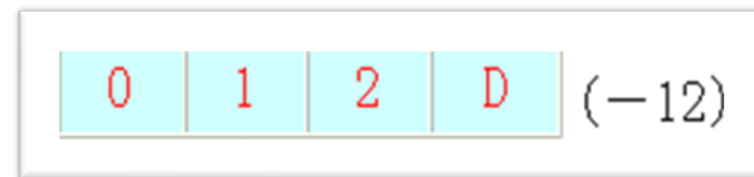
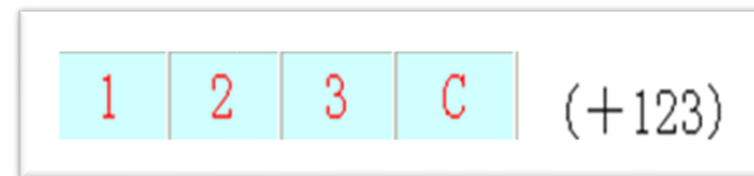
>>> 2.1.3 字符与字符串的表示方法 (1/2)

➤ 字符串形式

- 每个十进制**数位**占用一个字节；
- 除保存各数位，还需要指明该数存放的起始地址和总位数；
- 主要用于非数值计算的应用领域。

➤ 压缩的十进制数串形式

- 采用BCD码表示，一个字节可存放两个十进制数位；
- 节省存储空间，便于直接完成十进制数的算术运算；
- 用特殊的二进制编码表示数据正负，如1100—正、1101—负；



>>> 2.1.3 字符与字符串的表示方法 (2/2)

➤ ASCII码

(美国国家信息交换标准字符码)

- 包括128个字符，共需7位编码；
- ASCII码**最高位为0**，余下7位作为128个字符的编码。
- 最高位的作用
 - ✓ **奇偶校验；扩展编码。**

➤ 常用ASCII码如右表：

字符	ASCII码（十六进制表示）
‘0’~‘9’	30H ~ 39H
‘a’~‘z’	61H ~ 7AH
‘A’~‘Z’	41H ~ 5AH
空格	20H
‘\$’	24H
回车	0DH
换行	0AH

>>> 2.1.4 汉字的表示方法

➤ 汉字的输入编码

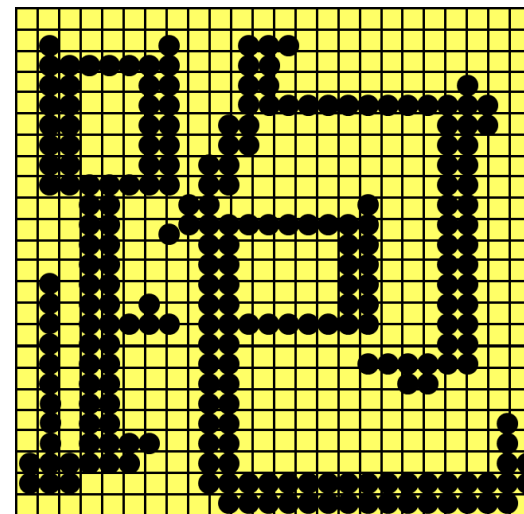
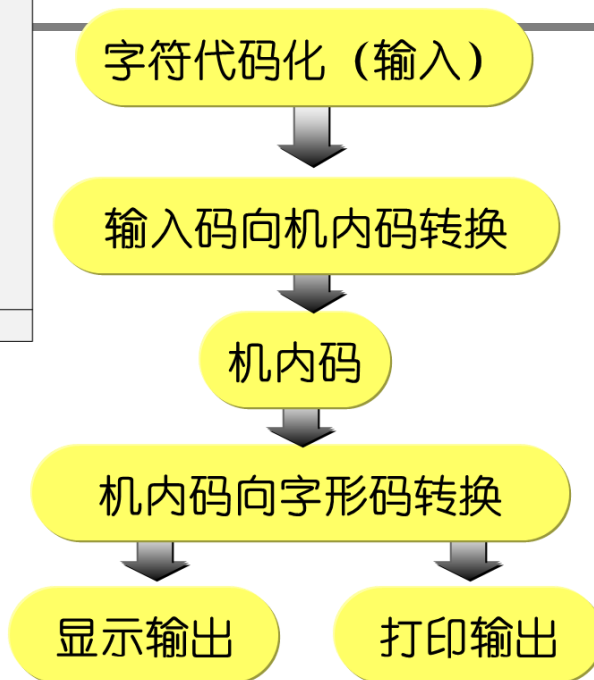
- 目的：直接使用西文标准键盘把汉字输入计算机。
- 分类：主要有数字编码、拼音码、字形编码三类。

➤ 汉字内码

- 用于汉字信息的存储、交换、检索等操作的机内代码。
- 如：汉字国标码GB2312、UNICODE编码、UTF编码等。
 - ✓GB2312：我国的“信息交换用汉字编码字符集”；
 - ✓Unicode：ISO制定，可容纳世界所有文字和符号的编码方案；
 - ✓UTF-8：以字节为单位对Unicode编码，字符长度不固定；

➤ 汉字字模码

- 点阵表示的汉字字形代码，用于汉字的输出。



>>> 2.1.5 校验码（数据校验）

- 数据校验：由数据特定编码形式，检查和纠正数据传送过程中出现的错误；
- 数据校验码（检错码）：能够发现某些错误或具有自动纠错能力的数据编码；
- 数据校验的原理：扩大码距；

□ 码距：任意两个合法码之间不同的二进制位的最少位数；

➤ 设有四位二进制编码

- 若表示16种状态，则任何一位出错，即变成另一个合法码；
 - ✓ 此时码距为1，无检错能力；
- 若表示8种状态（编码设置合适），则一位出错变为非法编码；
 - ✓ 此时码距可以扩大为2；
 - ✓ 注意：并不是任选8种编码都可扩大码距；

例如，
码距为2的4位编码
0000、0011、0101、
0110、1001、1010、
1100、1111

>>> 校验码的类型

➤ 奇偶校验码

- 根据数据中“1”的个数，设置1位校验位的值；
- 分奇校验和偶校验两种，只能检错，无纠错能力；

➤ 海明校验码

- 在奇偶校验的基础上，增加校验位而得；
- 具有检错和纠错的能力；

➤ 循环冗余校验码（CRC）

- 通过模2运算建立数据信息和校验位之间的约定关系；
- 具有很强的检错纠错能力。

数据中增加1个冗余位（奇偶校验位），使码距由1增加到2；
如果合法编码中有奇数位发生错误，即成为非法代码。

海明码 码距	编 码 能 力	
	检错	纠错
1	0	0
2	1	0
3	2 或 1	
4	2 并 1	
5	2 并 2	
6	3 并 2	
7	3 并 3	

海明码

□海明码：1950年提出

□在一个数据中加入几个校验位，**每个校验位和某几个特定的信息位构成偶校验的关系**；

□接收端对每个偶关系进行校验，产生校验因子；

□通过**校正因子**区分**无错**和码字中的 **n 个不同位置的错误**；

✓ 不同代码位上的错误会得出不同的校验结果；

➤海明码校验的过程

- ① 确定校验位的位数
- ② 确定校验位的位置
- ③ 校验分组
- ④ 校验位的形成
- ⑤ 接收端校验

● 设 **K** 为有效信息位数， **r** 为校验位位数，则整个编码的位数 **N** 应满足不等式：

○ $N = K + r \leq 2^r - 1$

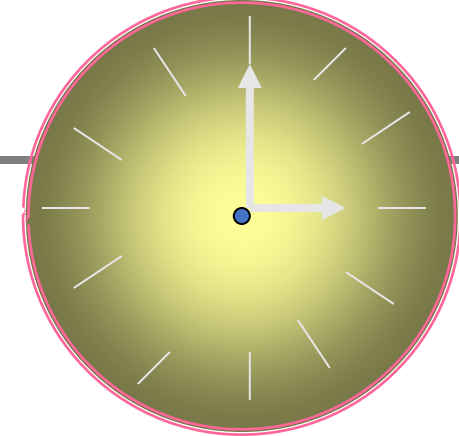
○ 通常称为 **(N, K) 海明码**

➤ 设有 **$(7, 4)$ 海明码**，则其编码长度为____位，校验位数为____位。

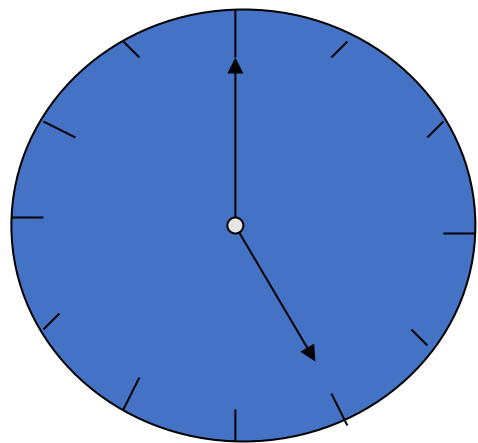
7

3

>>> 补码表示法的引入 (1/3)

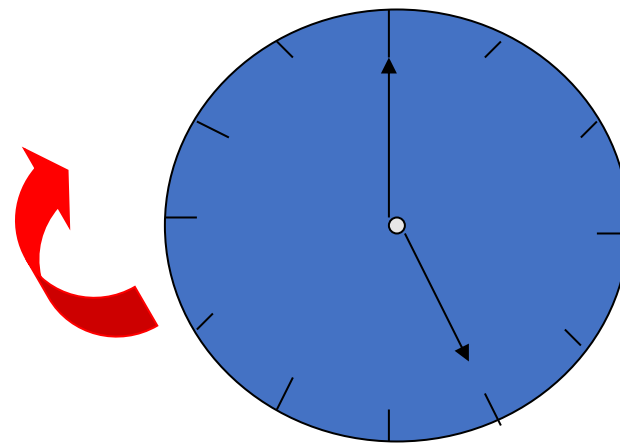


要将指向**5点**的时钟调整到**3点**整，应如何处理？



方式1：倒拨：

$$5-2=3$$



方式2，正拨：

$$5+10 \text{ (MOD) } 12 = 3$$

(12自动丢失。12就是模)

结论： 模：可以理解为一个计量系统的计量范围。例如时钟计量范围0~11，模=12。

在模**12**的系统下 $5-2=5+10 \text{ (MOD) } 12$

>>> 补码表示法的引入

➤ 继续推导:

$$\square 5 - 2 = 5 + 10 \pmod{12}$$

$$\square 5 + (-2) = 5 + 10 \pmod{12}$$

$$\square -2 = 10 \pmod{12} \quad \text{注意在模12的条件下}$$

➤ 结论:

在模为**12**的情况下，**-2**的补码就是**10**。一个负数用其补码代替，同样可以得到正确的运算结果。

>>>补码表示法的引入

➤进一步结论：

□在计算机中，机器能表示的数据位数是固定的，其运算都是有模运算。

✓若是 $n+1$ 位整数，则其模为 2^{n+1} ；

✓若是小数，则其模为2。

□若运算结果超出了计算机所能表示的数值范围，则只保留它的小于模的低 n 位的数值，超过 n 位的高位部分就自动舍弃了（联想纯小数补码绝对值最大的-1）。

>>>补码表示法----定义

➤定义:

□定点小数: $[x]_{\text{补}} = \begin{cases} x & 1 > x \geq 0 \\ 2+x = 2 - |x| & 0 \geq x \geq -1 \end{cases} \quad (\text{mod } 2)$

□定点整数: $[x]_{\text{补}} = \begin{cases} x & 2^n > x \geq 0 \\ 2^{n+1}+x = 2^{n+1}-|x| & 0 \geq x \geq -2^n \end{cases} \quad (\text{mod } 2^{n+1})$

➤举例: x为n+1位

□ $[+0.110]_{\text{补}} = 0.110$

□ $[-0.110]_{\text{补}} = 10 + (-0.110) = 1.010$

□ $[+110]_{\text{补}} = 0110$

□ $[-110]_{\text{补}} = 2^4 + (-110) = 10000 - 110 = 1010$

2.2 定点加法、减法运算

4.2.1 补码加法

4.2.2 补码减法

4.2.3 溢出概念与检验方法

4.2.4 基本的二进制加法、减法器

>>> 2.2.1 补码加法

教材P35以定点整数为例证明定义

➤ 补码加法运算基本公式

□ 定点整数: $[x+y]_{\text{补}} = [x]_{\text{补}} + [y]_{\text{补}} \pmod{2^{n+1}}$

定点小数: $[x+y]_{\text{补}} = [x]_{\text{补}} + [y]_{\text{补}} \pmod{2}$

➤ 证明

(1) 证明依据: 补码的定义(以定点小数为例)

$$[X]_{\text{补}} = \begin{cases} x & 1 > x \geq 0 \\ 2+x = 2 - |x| & 0 \geq x \geq -1 \end{cases} \pmod{2}$$

(2) 证明思路: 分三种情况。

(a) x 、 y 均为正值 ($x > 0, y > 0$)

(b) x 、 y 均为负值 ($x < 0, y < 0$)

(c) x 、 y 一正一负 ($x > 0, y < 0$ 或者 $x < 0, y > 0$)

证明:

(a) $x > 0, y > 0$

$$[x]_{\text{补}} + [y]_{\text{补}} = x + y = [x+y]_{\text{补}} \pmod{2}$$

(b) $x < 0, y < 0$

$$\because [x]_{\text{补}} = 2+x, [y]_{\text{补}} = 2+y$$

$$\begin{aligned} \therefore [x]_{\text{补}} + [y]_{\text{补}} &= 2+x+2+y = 2+(2+x+y) \\ &= 2+[x+y]_{\text{补}} \pmod{2} \\ &= [x+y]_{\text{补}} \end{aligned}$$

(c) $x > 0, y < 0$ ($x < 0, y > 0$ 的证明与此相同)

$$\because [x]_{\text{补}} = x, [y]_{\text{补}} = 2+y$$

$$\therefore [x]_{\text{补}} + [y]_{\text{补}} = x+2+y = 2+(x+y)$$

当 $x+y > 0$ 时, $2+(x+y) > 2$, 进位2必丢失;

因 $x+y > 0$, 故 $[x]_{\text{补}} + [y]_{\text{补}} = x+y = [x+y]_{\text{补}} \pmod{2}$

当 $x+y < 0$ 时, $2+(x+y) < 2$

$$\begin{aligned} \text{因 } x+y < 0, \text{ 故 } [x]_{\text{补}} + [y]_{\text{补}} &= 2+(x+y) \\ &= [x+y]_{\text{补}} \pmod{2} \end{aligned}$$

>>> 定点数补码加法举例

[例2.12] $x = +1001$, $y = +0101$,

求 $x + y$ 。

解:

$$[x]_{\text{补}} = 0\ 1001, [y]_{\text{补}} = 0\ 0101$$

$$\begin{array}{r} [x]_{\text{补}} \quad 0\ 1001 \\ + [y]_{\text{补}} \quad 0\ 0101 \\ \hline [x + y]_{\text{补}} \quad 0\ 1110 \end{array}$$

$$\therefore x + y = +1110$$

[例2.13] $x = +1011$, $y = -0101$,

求 $x + y$ 。

解:

$$[x]_{\text{补}} = 0\ 1011, [y]_{\text{补}} = 1\ 1011$$

$$\begin{array}{r} [x]_{\text{补}} \quad 0\ 1011 \\ + [y]_{\text{补}} \quad 1\ 1011 \\ \hline [x + y]_{\text{补}} \quad 10\ 0110 \end{array}$$

$$\therefore x + y = +0110$$

>>> 补码负的最小值的计算

$$[-127]_{\text{补}} = [-111\ 1111]_{\text{补}} = 1000\ 0001$$

$$[-1]_{\text{补}} = [-000\ 0001]_{\text{补}} = 1111\ 1111$$

$$11000\ 0000$$

$$\text{mod } 2^8$$

$$1000\ 0000 = [-128]_{\text{补}}$$

>>> 2.2.2 补码减法

➤ 补码减法运算基本公式

□ 定点整数: $[x - y]_{\text{补}} = [x]_{\text{补}} - [y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}} \pmod{2^{n+1}}$

□ 定点小数: $[x - y]_{\text{补}} = [x]_{\text{补}} - [y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}} \pmod{2}$

➤ 证明: 只需要证明 $[-y]_{\text{补}} = -[y]_{\text{补}}$

□ 已证明 $[x+y]_{\text{补}} = [x]_{\text{补}} + [y]_{\text{补}}$, 故 $[y]_{\text{补}} = [x+y]_{\text{补}} - [x]_{\text{补}}$

□ 又 $[x - y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}}$, 故 $[-y]_{\text{补}} = [x - y]_{\text{补}} - [x]_{\text{补}}$

□ 可得 $[y]_{\text{补}} + [-y]_{\text{补}} = [x+y]_{\text{补}} + [x - y]_{\text{补}} - [x]_{\text{补}} - [x]_{\text{补}}$
 $= [x + y + x - y]_{\text{补}} - [x]_{\text{补}} - [x]_{\text{补}}$
 $= [x + x]_{\text{补}} - [x]_{\text{补}} - [x]_{\text{补}} = 0$

➤ $[-y]_{\text{补}}$ 等于 $[y]_{\text{补}}$ 的各位取反, 末位加1。

>>> 定点数补码减法举例：课本P36 例2.14

已知 $x_1 = -1110$, $x_2 = +1101$,

求： $[x_1]_{\text{补}}$, $[-x_1]_{\text{补}}$, $[x_2]_{\text{补}}$, $[-x_2]_{\text{补}}$ 。

➤解：

$$[x_1]_{\text{补}} = 1\ 0010$$

$$\begin{aligned} [-x_1]_{\text{补}} &= \neg [x_1]_{\text{补}} + 1 \\ &= 0\ 1101 + 0\ 0001 = 0\ 1110 \end{aligned}$$

$$[x_2]_{\text{补}} = 0\ 1101$$

$$\begin{aligned} [-x_2]_{\text{补}} &= \neg [x_2]_{\text{补}} + 1 \\ &= 1\ 0010 + 0\ 0001 = 1\ 0011 \end{aligned}$$

注意课本这里
为 2^{-4} ，是将其
作为定点小数
处理！

>>> 定点数补码减法举例：课本P37 例2.15

$x = +1101$, $y = +0110$, 求 $x - y$ 。

➤解:

$$[x]_{\text{补}} = 0\ 1101, [y]_{\text{补}} = 0\ 0110, [-y]_{\text{补}} = 1\ 1010$$

$$[x - y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}}$$

$$= 0\ 1101 + 1\ 1010$$

$$= 10\ 0111$$

$$= 0\ 0111$$

$$\therefore x - y = +0111$$

$$\begin{array}{r} 0\ 1101 \\ +) 1\ 1010 \\ \hline \boxed{1}\ 0\ 0111 \end{array}$$

>>> 定点数补码加减法运算

➤ 基本公式

□ 定点整数:

$$[x \pm y]_{\text{补}} = [x]_{\text{补}} + [\pm y]_{\text{补}} \pmod{2^{n+1}}$$

□ 定点小数:

$$[x \pm y]_{\text{补}} = [x]_{\text{补}} + [\pm y]_{\text{补}} \pmod{2}$$

➤ 定点数补码加减法运算

□ 符号位和数值位可同等处理;

□ 只要结果不溢出, 将结果按 2^{n+1} 或2取模, 即为本次运算结果。

例. 设机器字长为8位, $[x]_{\text{补}} = 1010\ 0011$, $[y]_{\text{补}} = 0010\ 1101$,
求 $x - y$ 。

作答

>>> 例 设机器字长为8位, $[x]_{\text{补}} = 1010\ 0011$, $[y]_{\text{补}} = 0010\ 1101$,
求 $x - y$.

$x = -93$, $y = +45$

解:

$$[-y]_{\text{补}} = 1101\ 0011$$

$$[x - y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}}$$

$$= 1010\ 0011 + 1101\ 0011$$

$$= 1\ 0111\ 0110$$

$$= 0111\ 0110$$

$$\therefore x - y = +118$$

$$\begin{array}{r} 1010\ 0011 \\ +) 1101\ 0011 \\ \hline 1\ 0111\ 0110 \end{array}$$

计算过程中, 产生了溢出!

$$-93 - 45 = -138 < -128$$

>>> 2.2.3 溢出概念与检测方法

➤ 溢出

□ 定点数机中，数值大小超出数据所能表示的范围。

➤ 正溢（上溢）

□ 数据大于机器所能表示的最大正数；

➤ 负溢（下溢）

□ 数据小于机器所能表示的最小负数；

➤ 例如，4位补码表示的定点整数，范围为[-8, +7]

□ 若 $x = 5$, $y = 4$, 则 $x+y$ 产生上溢

□ 若 $x = -5$, $y = -4$, 则 $x+y$ 产生下溢

□ 若 $x = 5$, $y = -4$, 则 $x-y$ 产生上溢

例. $x = +0.1011$, $y = +0.1001$,

求 $x + y$ 。

[解:] $[x]_{\text{补}} = 0.1011$

$[y]_{\text{补}} = 0.1001$

$[x]_{\text{补}} \quad 0.1011$

$+ [y]_{\text{补}} \quad 0.1001$

$[x+y]_{\text{补}} \quad 1.0100$

正数+正数=负数

溢出!

例. $x = -0.1101$, $y = -0.1111$,

求 $x + y$ 。

[解:] $[x]_{\text{补}} = 1.0011$

$[y]_{\text{补}} = 1.0001$

$[x]_{\text{补}} \quad 1.0011$

$+ [y]_{\text{补}} \quad 1.0001$

$[x+y]_{\text{补}} \quad 0.0101$

负数+负数=正数

>>> 溢出判别方法1——直接判别法

➤ 方法:

□ 同号补码相加, 结果符号位与加数相反;

□ 异号补码相减, 结果符号位与减数相同;

➤ 硬件实现较复杂: x 和 y 的符号位异或后, 再与结果符号位异或;

➤ 举例:

□ 若 $[x]_{\text{补}}=0101$, $[y]_{\text{补}}=0100$, 则 $[x+y]_{\text{补}}=1001$ 上溢

□ 若 $[x]_{\text{补}}=1011$, $[y]_{\text{补}}=1100$, 则 $[x+y]_{\text{补}}=0111$ 下溢

□ 若 $[x]_{\text{补}}=0101$, $[y]_{\text{补}}=1100$, 则 $[x-y]_{\text{补}}=1001$ 上溢

>>> 溢出判别方法2——变形补码判别法(双符号位)

➤ 变形补码（模4补码）：

□ 采用双符号位表示补码，00-正数，11-负数；

□ 若运算结果双符号位相反，则溢出；

✓ 01：正溢，大于最大正数；

✓ 10：负溢，小于最小负数；

双符号位	结果
00	正
01	正溢
10	负溢
11	负

➤ 优点：硬件实现简单，只需对结果的双符号位进行异或；

➤ 举例：

□ 若 $[x]_{\text{补}} = 00101$ ， $[y]_{\text{补}} = 00100$ ，则 $[x+y]_{\text{补}} = 01001$

正溢

□ 若 $[x]_{\text{补}} = 11011$ ， $[y]_{\text{补}} = 11100$ ，则 $[x+y]_{\text{补}} = 10111$

负溢

□ 若 $[x]_{\text{补}} = 00101$ ， $[y]_{\text{补}} = 11100$ ，则 $[x-y]_{\text{补}} = 01001$

正溢

>>> 溢出判别方法3——进位判别法（单符号）

➤ 判别方法：

□ 最高数值位的进位与符号位的进位是否相同；

➤ 判别公式

□ 溢出标志 $OF = C_f \oplus C_{n-1}$

✓ C_f 为符号位产生的进位；

✓ C_{n-1} 为最高数值位产生的进位。

➤ 硬件实现简单，只需对两个进位位进行异或；

➤ 现代处理器采用这种方法

$$\begin{array}{r} 0101 \\ +) 0100 \\ \hline \end{array} \quad OF=0\oplus 1=1$$

1001
↑↑
0 1

$$\begin{array}{r} 0001 \\ +) 0100 \\ \hline \end{array} \quad OF=0\oplus 0=0$$

0101
↑↑
0 0

2014年考研统考题目 第13题

若 $x=103$ ， $y=-25$ ，则下列表达式采用8位定点补码运算实现时，会发生溢出的是（ ）。

☐ A $x+y$

☐ B $-x+y$

☒ C $x-y$

☐ D $-x-y$

提交

2014年考研统考题目 第13题

若 $x=103$ ， $y=-25$ ，则下列表达式采用8位定点补码运算实现时，会发生溢出的是（ ）。

解：同号相加或异号相减才有可能溢出，排除A和D

8位定点整数范围-128----- +127

A $x+y$

B $-x+y$

C $x-y$

D $-x-y$

2018年考研统考题目 第13题

假定带符号整数采用补码表示，若int型变量x和变量y的机器数分别是FFFF FFDFH和0000 0041H，则x、y的值以及x-y的机器数分别是___。

- ☐ A x=-65, y=41, x-y的机器数溢出
- ☐ B x=-33, y=65, x-y的机器数为FFFF FF9DH
- ☒ C x=-33, y=65, x-y的机器数为FFFF FF9EH
- ☐ D x=-65, y=41, x-y的机器数为FFFF FF96H

提交

减法指令“sub R1, R2, R3”的功能为“(R1)-(R2)→(R3)”，该指令执行后将生成进/借位标志CF和溢出标志OF。

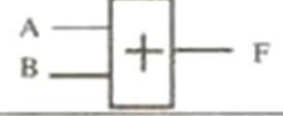
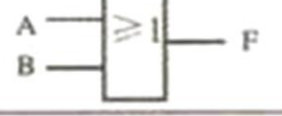
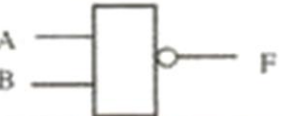
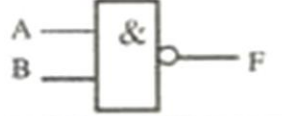
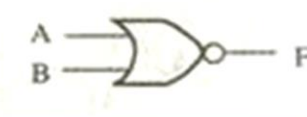
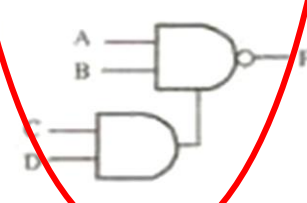
若(R1)= FFFF FFFFH, (R2)= FFFF FFF0H, 则减法指令执行后, CF与OF分别为 ()。

- ☒ A CF=0, OF=0
 ☐ B CF=1, OF=0
 ☐ C CF=0, OF=1
 ☐ D CF=1, OF=1

进/借位标志CF按加减数据的大小设置;
溢出标志OF按溢出规则设置。

提交

回顾逻辑门图形符号

逻辑门	IEEE 推荐符号	我国部颁符号	输出表达式	标准符号
“与”门			$F = A \cdot B$	
“或”门			$F = A + B$	
“非”门			$F = \bar{A}$	
“与非”门			$F = \overline{A \cdot B}$	
“或非”门			$F = \overline{A + B}$	
“异或”门			$F = A \oplus B$ $= A \bar{B} + \bar{A} B$	
“与或非”门			$F = \overline{AB + CD}$	

>>> 2.2.4 基本的二进制加法/减法器

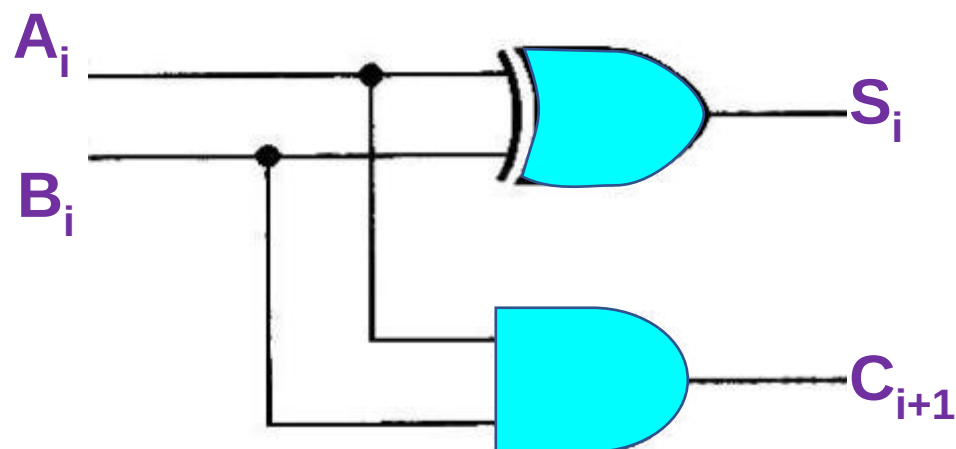
□一位二进制数据的半加器:

□加数: A_i 、 B_i

□结果: S_i (和)

C_{i+1} (本位向高位的进位)

□一位半加器逻辑图



输入			输出	
A_i	B_i		S_i	C_{i+1}
0	0		0	0
0	1		1	0
1	0		1	0
1	1		0	1

➤ $S_i = A_i \oplus B_i$

➤ $C_{i+1} = A_i B_i$

➤ 两输入，两输出，不考虑进位

>>> 一位二进制数据的全加器的逻辑结构

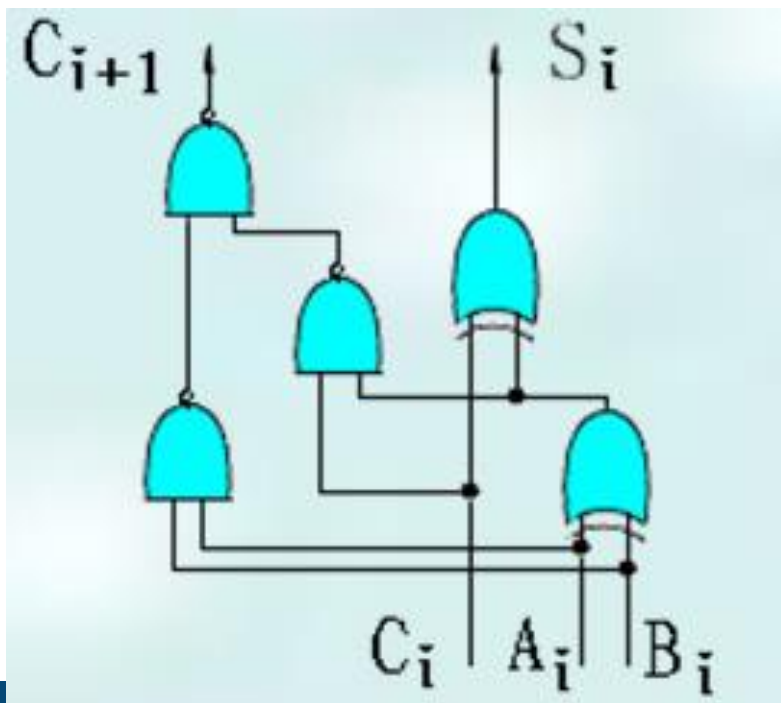
➤ 全加运算的真值表如右所示：

➤ 两个输出端的逻辑表达式

$$\square S_i = A_i \oplus B_i \oplus C_i$$

$$\square C_{i+1} = A_i B_i + B_i C_i + C_i A_i = A_i B_i + (A_i \oplus B_i) C_i$$

➤ 全加器逻辑结构



输入			输出	
A_i	B_i	C_i	S_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

2.2.4 基本的二进制加法/减法器

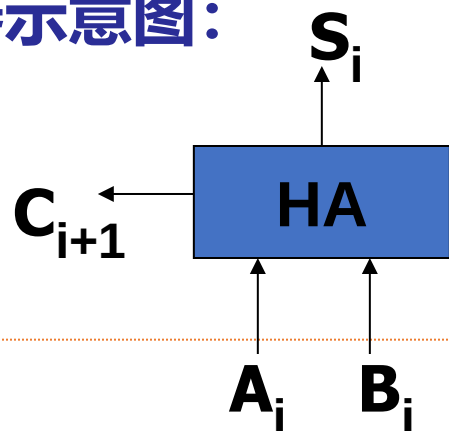
➤一位二进制数据的半加器:

□加数: A_i 、 B_i

□结果: S_i (和)

C_{i+1} (本位向高位的进位)

➤一位半加器示意图:



➤一位二进制数据的全加器:

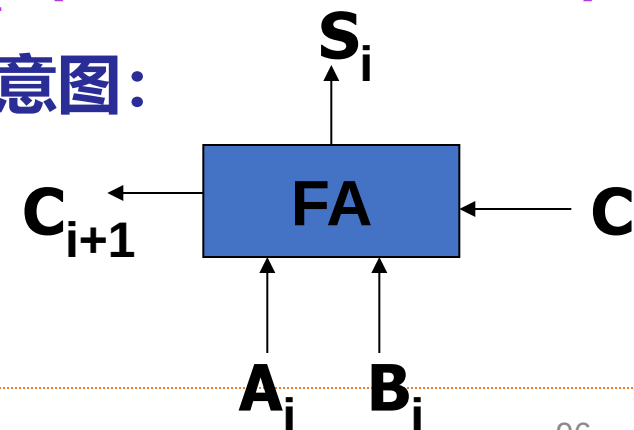
□加数: A_i 、 B_i

C_i (低位向本位的进位)

□结果: S_i (和)

C_{i+1} (本位向高位的进位)

➤一位全加器示意图:



>>> 多位二进制数据加法器

➤ 两个n位的数据

$$\square A = A_{n-1}A_{n-2}\cdots A_1A_0$$

$$\square B = B_{n-1}B_{n-2}\cdots B_1B_0$$

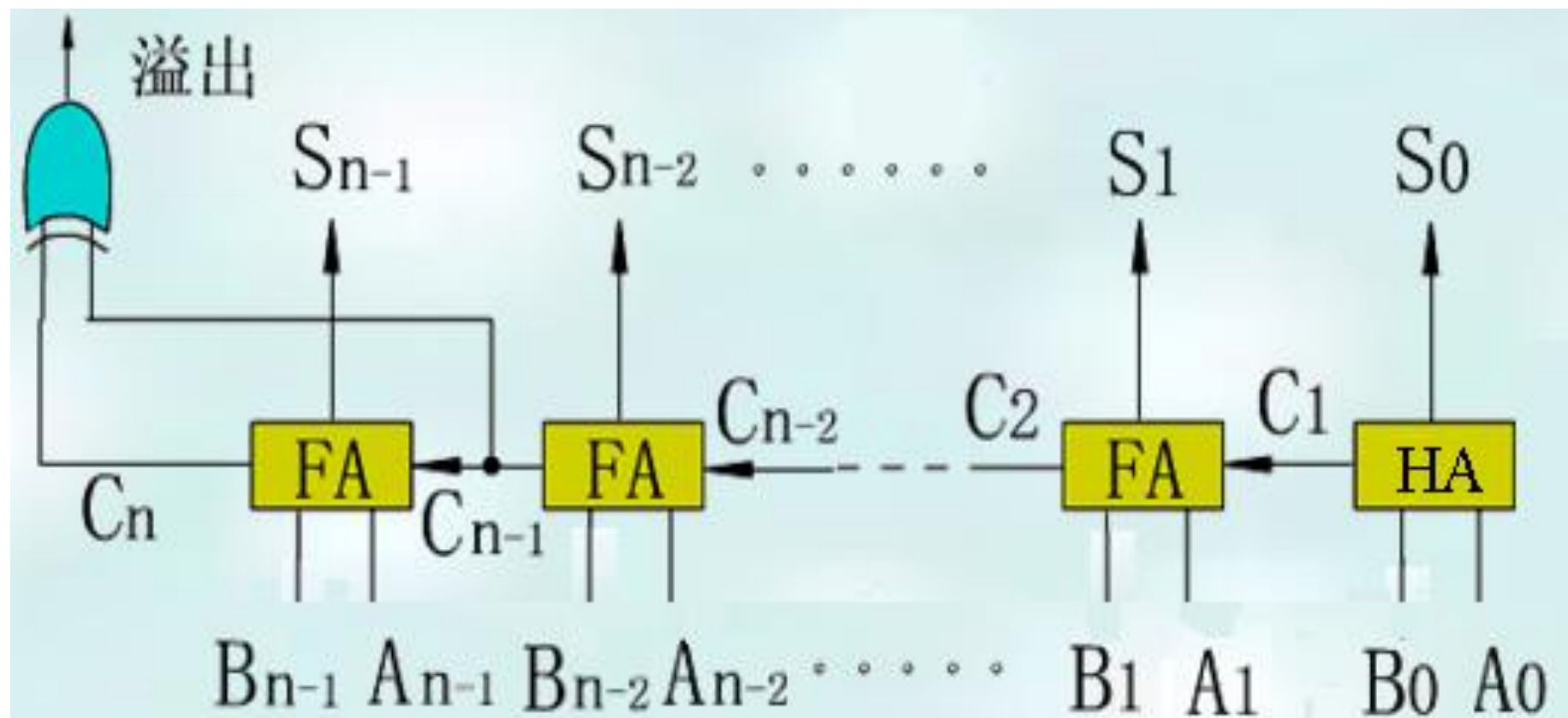
➤ 结果和

$$\square S = S_{n-1}S_{n-2}\cdots S_1S_0$$

➤ 溢出判断

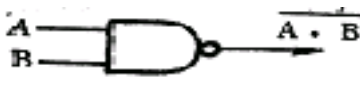
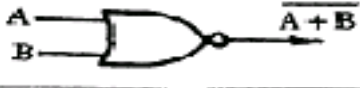
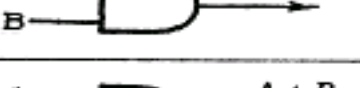
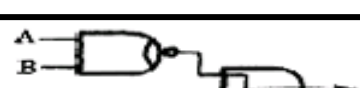
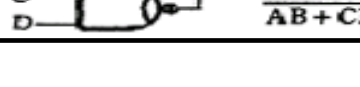
□ 进位判别法

$$\square V = C_n \oplus C_{n-1}$$





典型门电路的逻辑符号和延迟时间

门的名称	门的功能	逻辑符号（正逻辑）	时间延迟
与非	NAND		T
或非	NOR		T
非	NOT		T
与	AND		2T
或	OR		2T
异或	XOR		3T
异或非	XNOR		3T
接线逻辑 (与或非)	AOI		$T + T_{RC}$

T被定义为相应于单级逻辑电路的单位门延迟。

T通常采用一个“与非”门或一个“或非”门的时间延迟来作为度量单位。

>>> 延迟时间

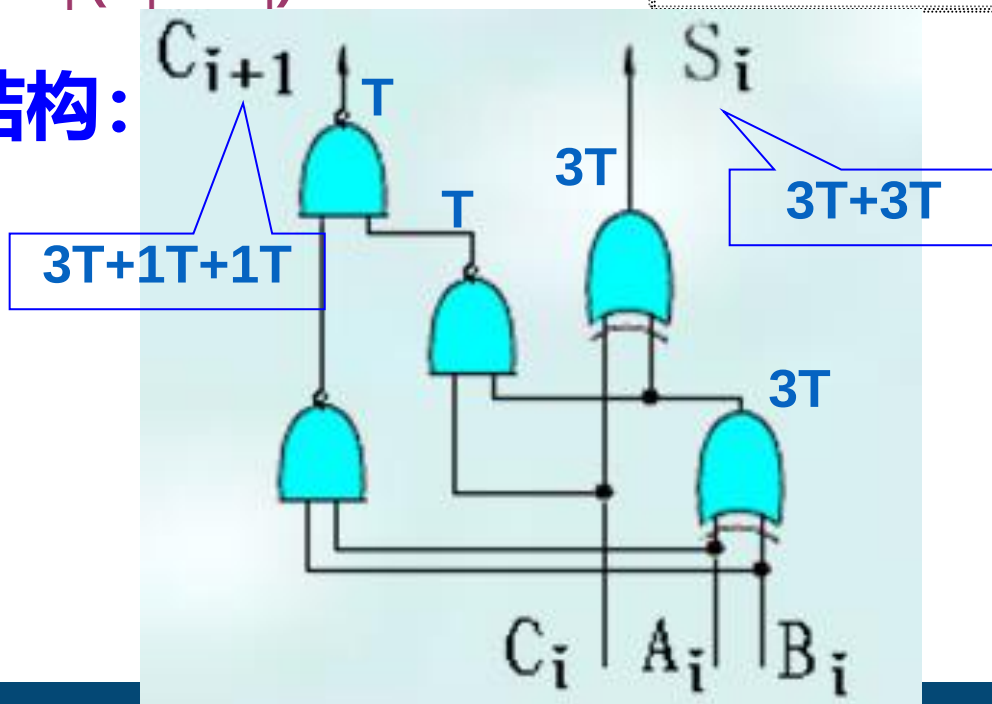
➤ 全加运算的真值表如右所示:

➤ 两个输出端的逻辑表达式

$$\square S_i = A_i \oplus B_i \oplus C_i$$

$$\square C_{i+1} = A_i B_i + C_i (B_i + A_i)$$

➤ 全加器逻辑结构:

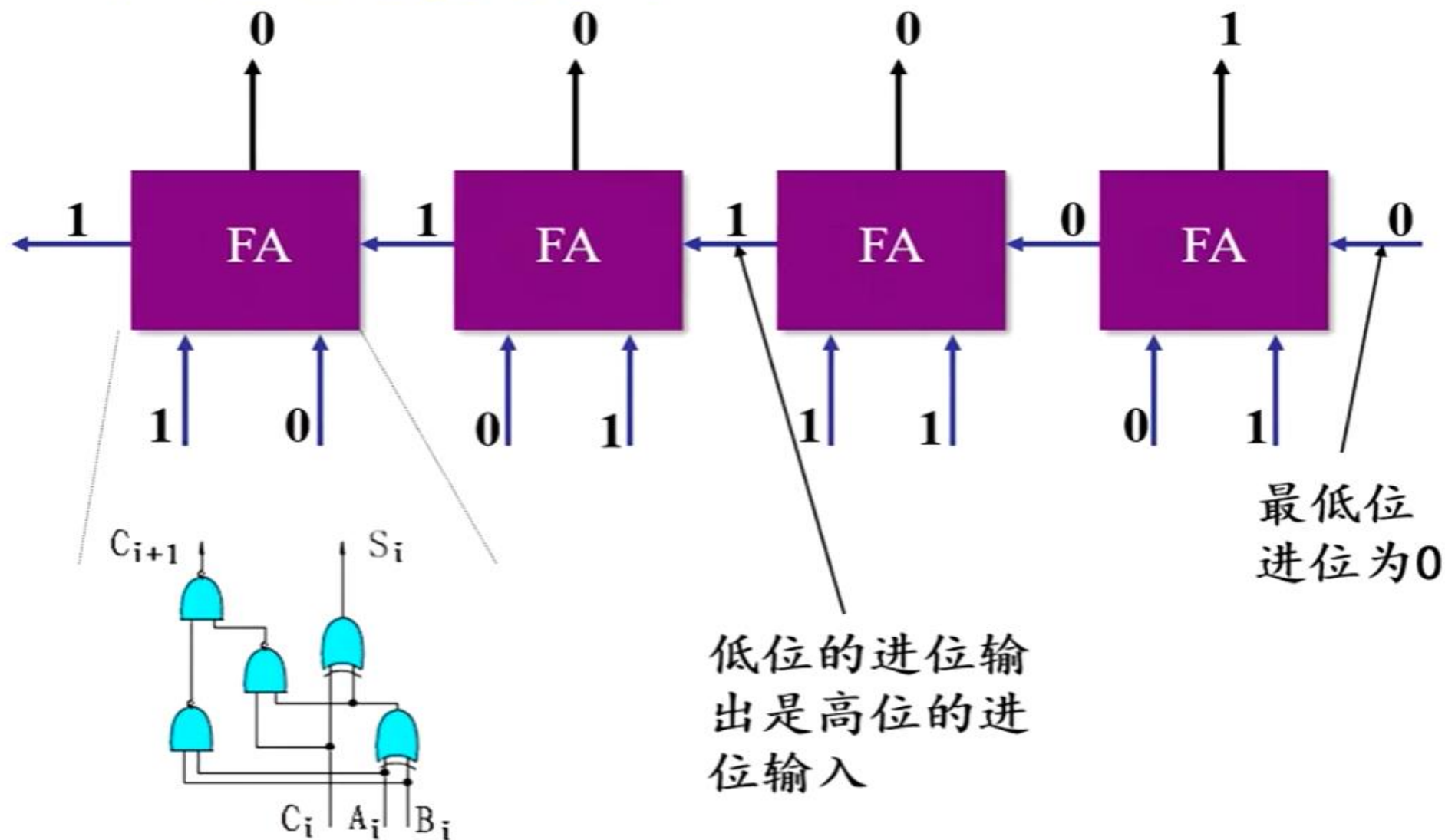


延迟时间

- 与门、非门延迟时间 $1T$,异或门延迟时间 $3T$
- 由 A_i 、 B_i 到 S_i 的延迟 时间为 $6T$
- 由 A_i 、 B_i 到 C_{i+1} 到的延迟 时间为 $5T$
- 由 C_i 到 C_{i+1} 的延迟时间为 $2T$



4位加法器设计



>>> 多位二进制数据加法/减法器

➤ 将减法转换成加法

$$\square [A]_{\text{补}} - [B]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}}$$

➤ 由 $[B]_{\text{补}}$ 求 $[-B]_{\text{补}}$

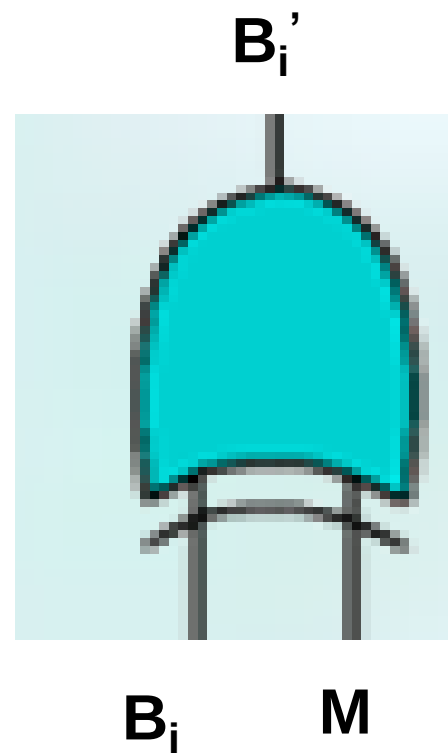
$\square [B]_{\text{补}}$ 求各位取反，末位加1；

➤ 将加减法电路合二为一

$\square$ 使用异或运算；

$\square$ 当 $M=0$ 时， $B_i' = B_i$

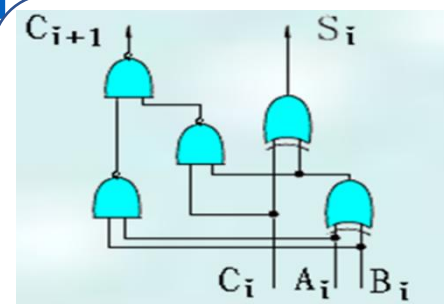
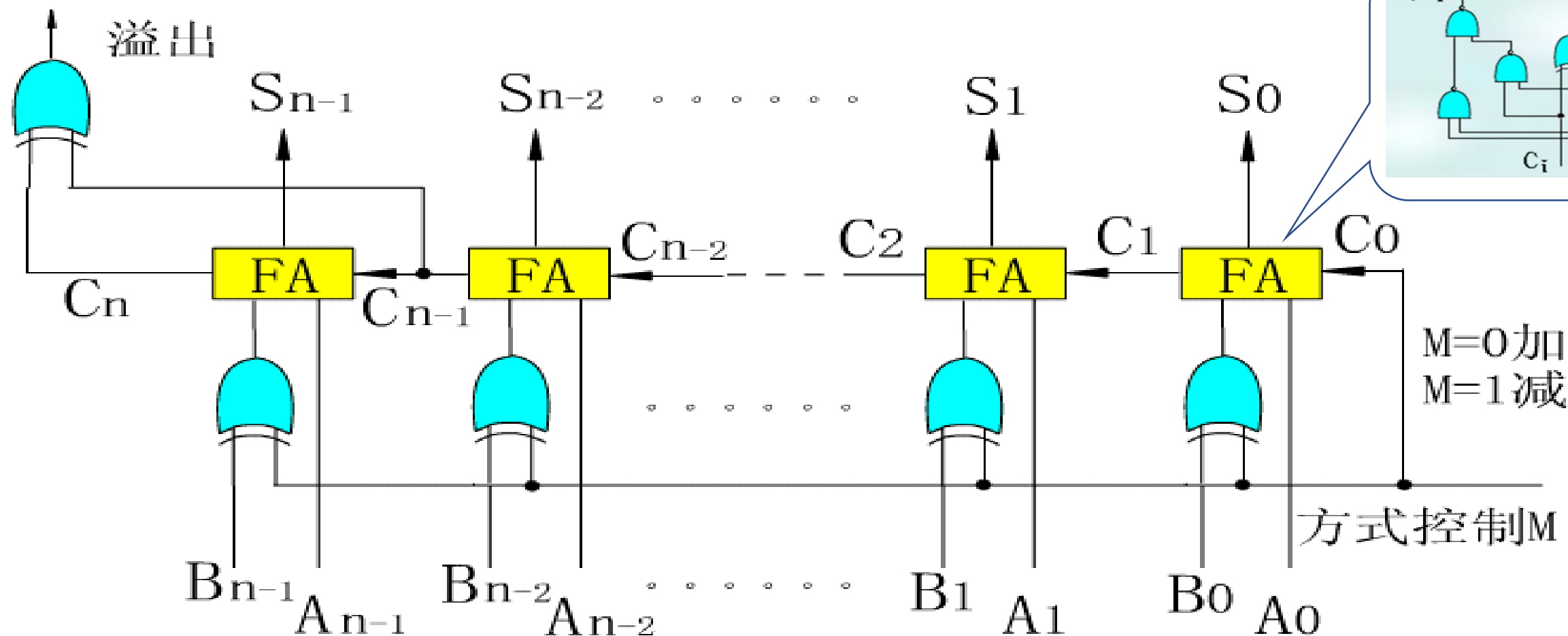
$\square$ 当 $M=1$ 时， $B_i' = \neg B_i$ ；



>>> n位行波进位的加法/减法器

动画演示:

2-1. swf



符号位

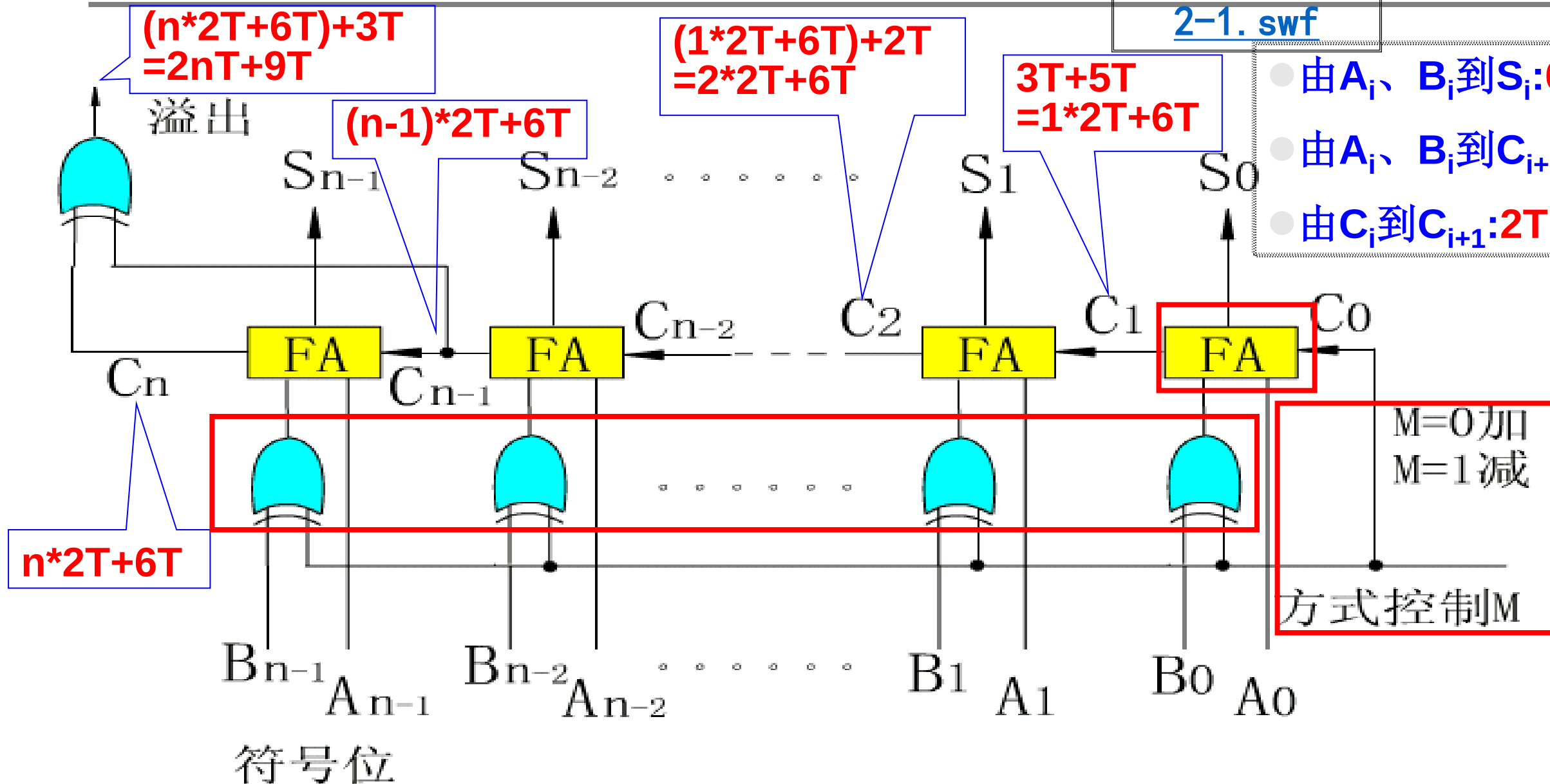
$$[A]_{\text{补}} - [B]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}}$$

>>> n位行波进位的加法/减法器

动画演示:

2-1. swf

- 由 A_i 、 B_i 到 S_i : $6T$
- 由 A_i 、 B_i 到 C_{i+1} : $5T$
- 由 C_i 到 C_{i+1} : $2T$



>>> n位行波进位加法/减法器的输出延迟

- 假如每位均采用一位全加器并考虑溢出检测，n位行波进位加法器的延迟时间 t_a 为：

$$t_a = n * 2T + 9T = (2n + 9)T$$

- 如果不考虑溢出，则延迟时间 t_a 由 S_{n-1} 的输出延迟决定：

$$\begin{aligned} t_a &= (n-1) * 2T + 6T + 3T \\ &= (2(n-1) + 9)T \end{aligned}$$

- 延迟时间 t_a

- 输入稳定后，在最坏情况下加法器得到稳定的输出所需的最长时间。
- 显然这个时间越小越好。

2.3 定点乘法运算

2.3.0 串行乘法

2.3.1 原码并行乘法

2.3.2 直接补码并行乘法

2.3.0 串行乘法

1. 分析笔算乘法

$$A = -0.1101 \quad B = 0.1011$$

$$A \times B = \textcircled{-}0.10001111 \quad \text{乘积的符号心算求得}$$

$$\begin{array}{r} 0.1101 \\ \times 0.1011 \\ \hline 1101 \\ 1101 \\ 0000 \\ 1101 \\ \hline 0.10001111 \end{array}$$

- ✓ 符号位单独处理
- ✓ 乘数的某一位决定是否加被乘数
- ? 4个位积一起相加
- ✓ 乘积的位数扩大一倍

2. 笔算乘法的逻辑实现

$$A \cdot B = A \cdot 0.1011$$

$$= 0.1A + 0.00A + 0.001A + 0.0001A$$

$$= 0.1A + 0.00A + 0.001(A + 0.1A)$$

$$= 0.1A + 0.01[0 \cdot A + 0.1(A + 0.1A)]$$

右移一位 $= 0.1\{A + 0.1[0 \cdot A + 0.1(A + 0.1A)]\}$

$$= 2^{-1} \{ A + 2^{-1} [0 \cdot A + 2^{-1} (A + 2^{-1} (A + 0))] \}$$

第一步 被乘数 $A + 0$

第二步 部分积右移1位，得新的部分积

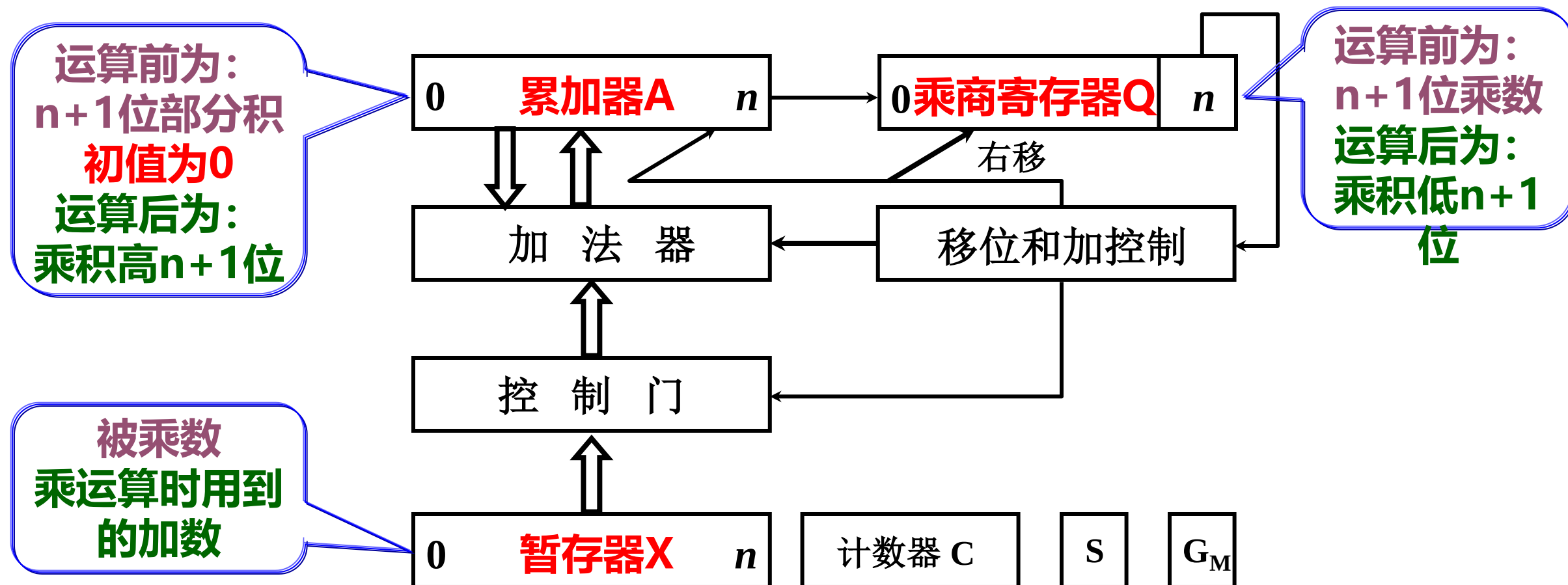
第三步 部分积 + 被乘数

•
•
•

第八步 部分积右移1位，得结果

$$\begin{array}{r}
 0.1101 \\
 \times 0.1011 \\
 \hline
 1101 \\
 1101 \\
 0000 \\
 1101 \\
 \hline
 0.10001111
 \end{array}$$

串行乘法的硬件配置



A、X、Q 均 $n+1$ 位, 移位和加受末位乘数控制

>>> 小结

➤ 乘法运算 = 加法 + 移位。

□ 若乘数数值位 $n = 4$ ，则累加 4 次，移位 4 次；

➤ 硬件构成

□ 多位数据的**加法器**；

□ 一个寄存器（**暂存器**）：存放被乘数，运算前后不变；

□ 两个具有移位功能的寄存器：**累加器**、**乘商寄存器**

✓ 运算前一个存放乘数，一个清空，准备做累加；

✓ 运算后保存乘积（位数为乘数的2倍）；

□ **加法和移位控制**：

✓ 根据乘数的每一位决定乘数是否送入加法器；

✓ 完成一次加法后，累加器和乘商寄存器同时右移一位；

□ **控制门**：由乘数数位控制，将暂存器的乘数送入加法器，或将0送入；

2013年考研统考题目 第14题

某字长为8位的计算机中，已知整型变量x、y的机器数分别为

$$[x]_{\text{补}} = 1111\ 0100$$

$$[y]_{\text{补}} = 1011\ 0000$$

若整型变量 $z = 2 * x + y / 2$ ，则z的机器数为（ ）



A 1 1000000



B 0010 0100



C 1010 1010



D 溢出

提交

>>> 2.3.1 原码并行乘法

➤ 不带符号的阵列乘法器

两个无符号数据的并行乘法电路

□ 设有两个不带符号的二进制整数：

$$A = a_{m-1} \cdots a_1 a_0 \quad ; \quad B = b_{n-1} \cdots b_1 b_0$$

□ 它们的数值分别为a和b，即

$$a = \sum_{i=0}^{m-1} a_i 2^i \quad b = \sum_{j=0}^{n-1} b_j 2^j$$

□ A、B相乘，产生m + n位乘积P：

$$P = p_{m+n-1} \cdots p_1 p_0$$

□ 乘积P 的数值为

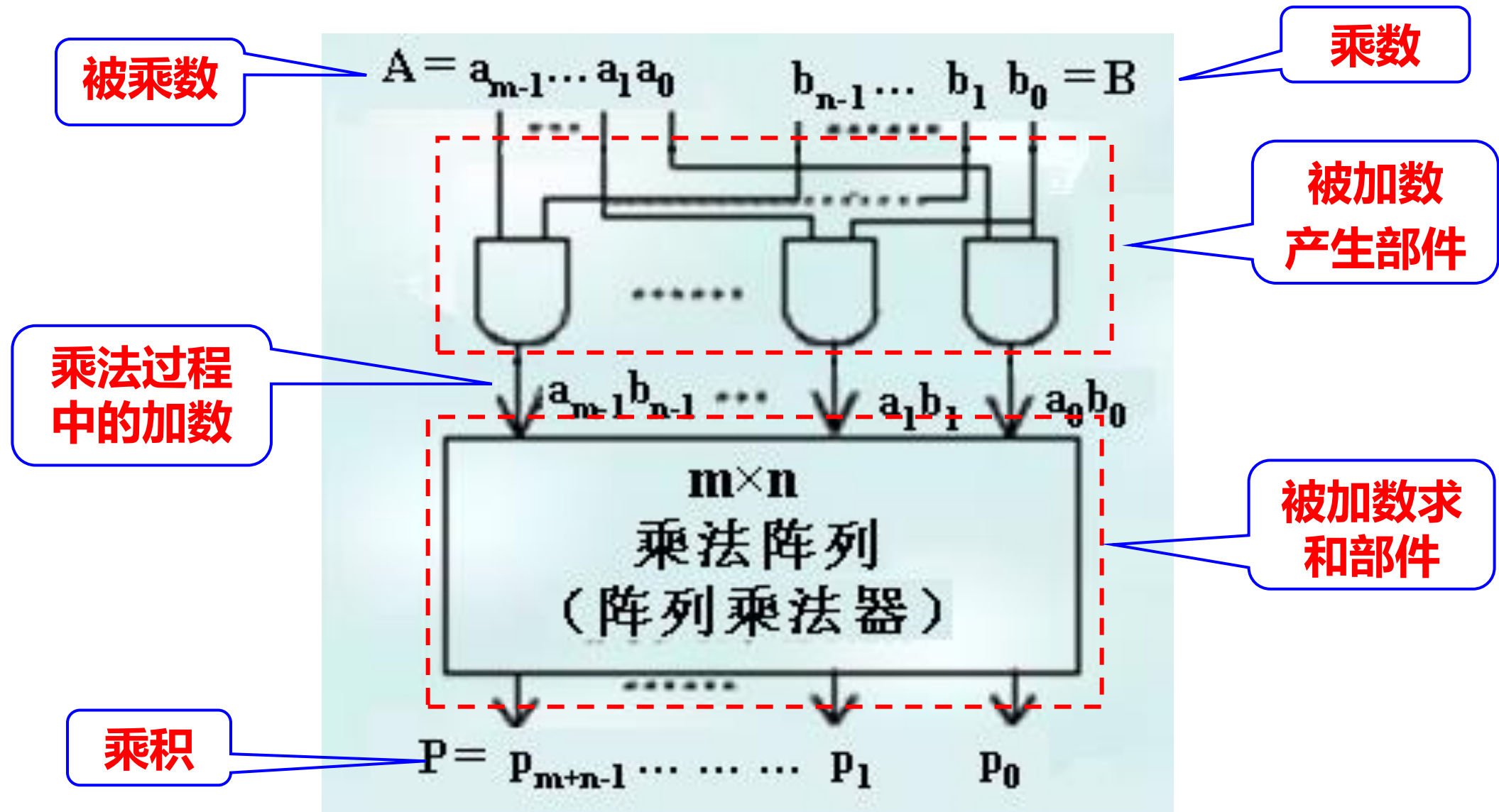
$$p = ab = \left(\sum_{i=0}^{m-1} a_i 2^i \right) \left(\sum_{j=0}^{n-1} b_j 2^j \right) = \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} (a_i b_j) 2^{i+j} = \sum_{k=0}^{m+n-1} p_k 2^k$$

二进制数乘法的运算过程

$$\begin{array}{r}
 \begin{array}{ccccccc}
 & & & a_{m-1} & a_{m-2} & \cdots & a_1 & a_0 = A \\
 \times & & & & & & b_{n-1} & \cdots & b_1 & b_0 = B \\
 \hline
 & & & a_{m-1}b_0 & a_{m-2}b_0 & \cdots & a_1b_0 & a_0b_0 \\
 & & & a_{m-1}b_1 & a_{m-2}b_1 & \cdots & a_1b_1 & a_0b_1 \\
 & & & \vdots & \vdots & & \vdots & \vdots \\
 & & & a_{m-1}b_{n-1} & a_{m-2}b_{n-1} & \cdots & a_1b_{n-1} & a_0b_{n-1} \\
 \hline
 p_{m+n-1} & p_{m+n-2} & p_{m+n-3} & \cdots & p_{n-1} & \cdots & p_1 & p_0 = P
 \end{array}
 \end{array}$$



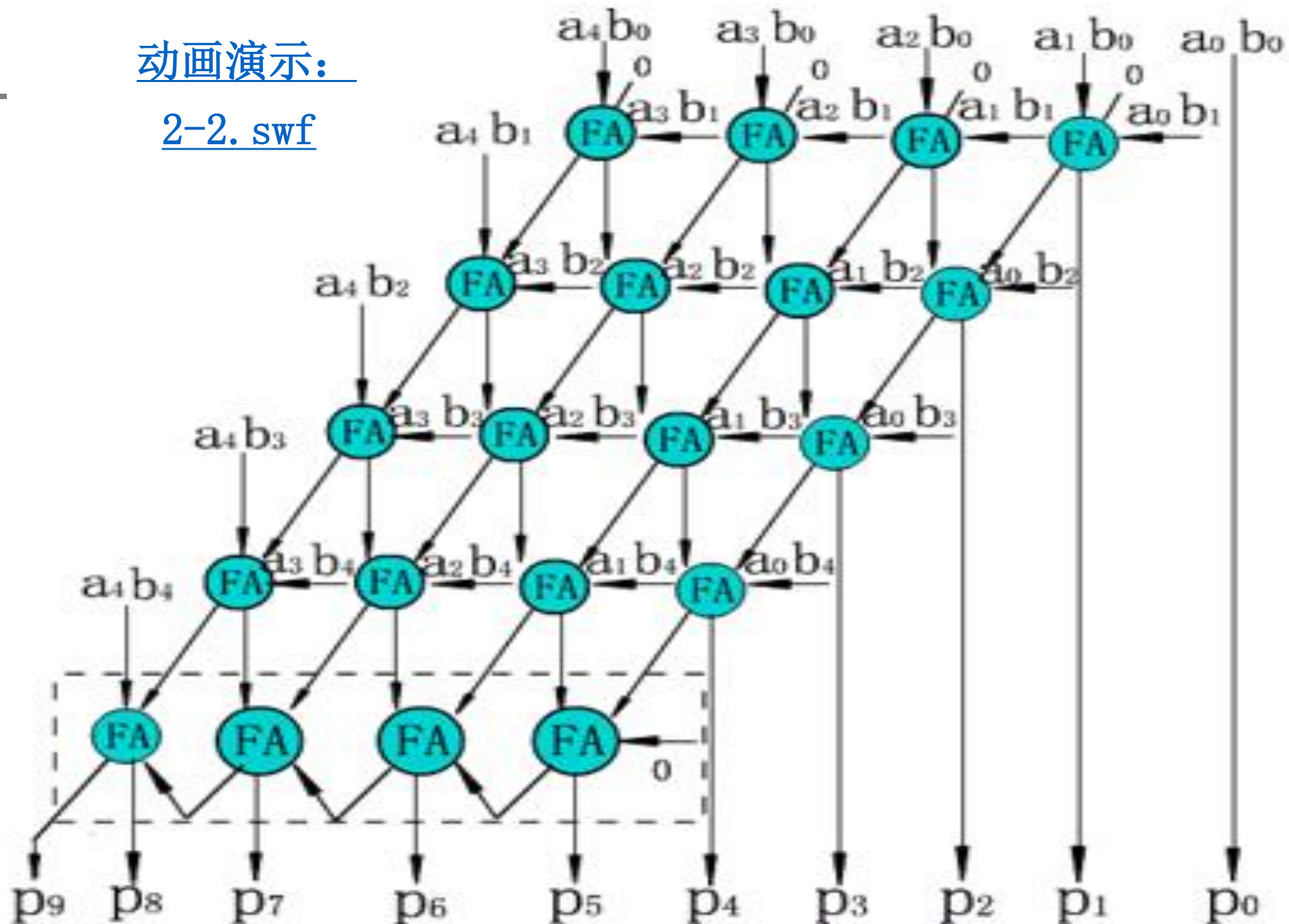
$m \times n$ 位不带符号的阵列乘法器逻辑框图





5 × 5 位不带符号阵列乘法器电路

动画演示:
[2-2. swf](#)



>>> 3.带符号的阵列乘法器

➤带符号原码的乘法:

□设有两个 $n+1$ 位带符号的二进制整数:

$$A = a_n \dots a_1 a_0 \quad ; \quad B = b_n \dots b_1 b_0$$

□乘积 $A*B=(a_n \oplus b_n)(a_{n-1} \dots a_1 a_0 * b_{n-1} \dots b_1 b_0)$

➤带符号数的阵列乘法器间接补码乘法电路

□在无符号数的阵列乘法器的基础上完成;

□使用3个求补器, 分别用对应的符号位控制;

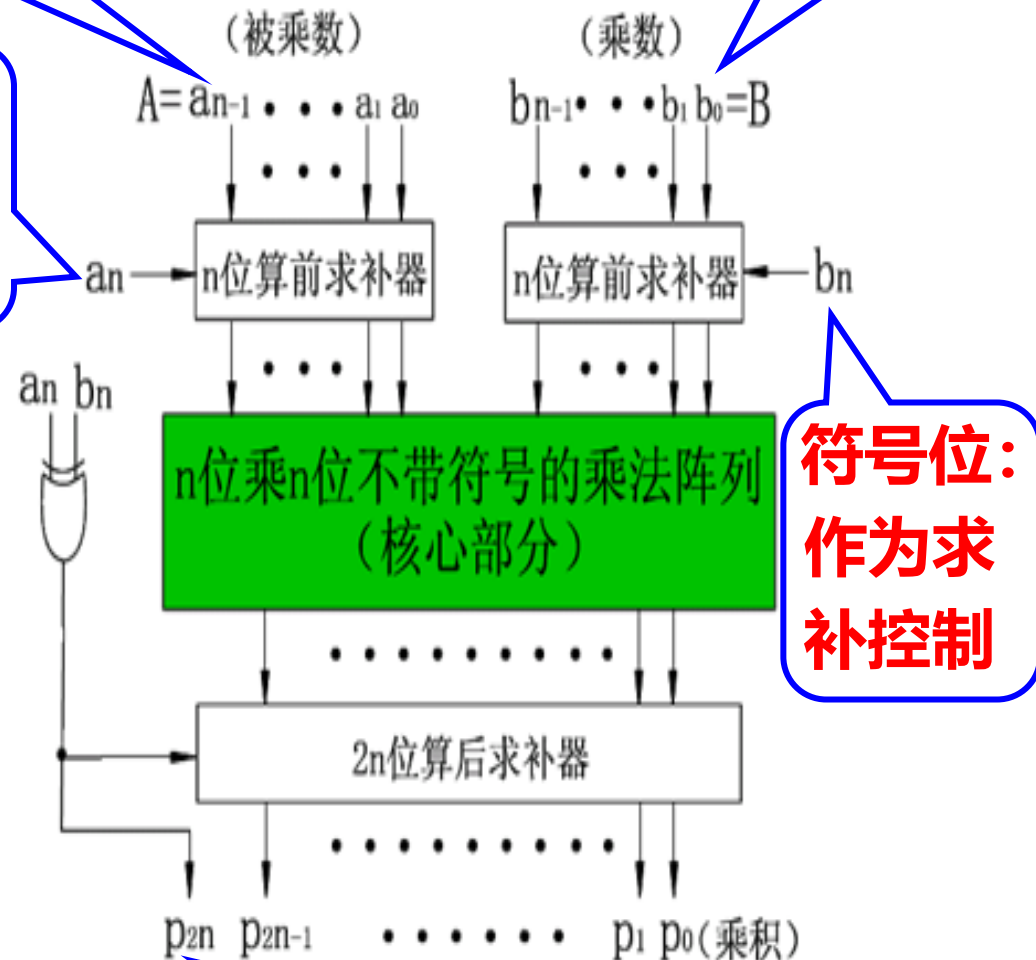
✓ 2个算前求补器: 将乘数真值的绝对值送入运算器;

✓ 1个算后求补器: 将负的乘积的真值求出补码表示;

符号位:
作为求
补控制

补码数值

补码数值



符号位:
作为求
补控制

结果符号位:
同时控制结果的求补

>>> 对2求补电路的原理

➤ 设有两个 $n+1$ 位带符号补码： $A = a_n \dots a_1 a_0$; $B = b_n \dots b_1 b_0$

➤ 采用原码乘法电路计算 $A*B$

□ 符号位不参与运算，只用于确定结果的符号；

□ 数据位要取绝对值参与运算：正数——不变；负数——各位取反，末位加1

➤ 对2求补电路：

□ 数据举例：

$[-2]_{\text{补}} = 1\overset{\text{---}}{\underset{\text{▲}}{1}}10$	$[-3]_{\text{补}} = 1\overset{\text{---}}{\underset{\text{▲}}{1}}01$	$[-6]_{\text{补}} = 1\overset{\text{---}}{\underset{\text{▲}}{0}}10$
$[+2]_{\text{补}} = 10\overset{\text{---}}{\underset{\text{▲}}{1}}0$	$[+3]_{\text{补}} = 00\overset{\text{---}}{\underset{\text{▲}}{1}}1$	$[+6]_{\text{补}} = 0\overset{\text{---}}{\underset{\text{▲}}{1}}10$

□ 两相反数的补码特征：

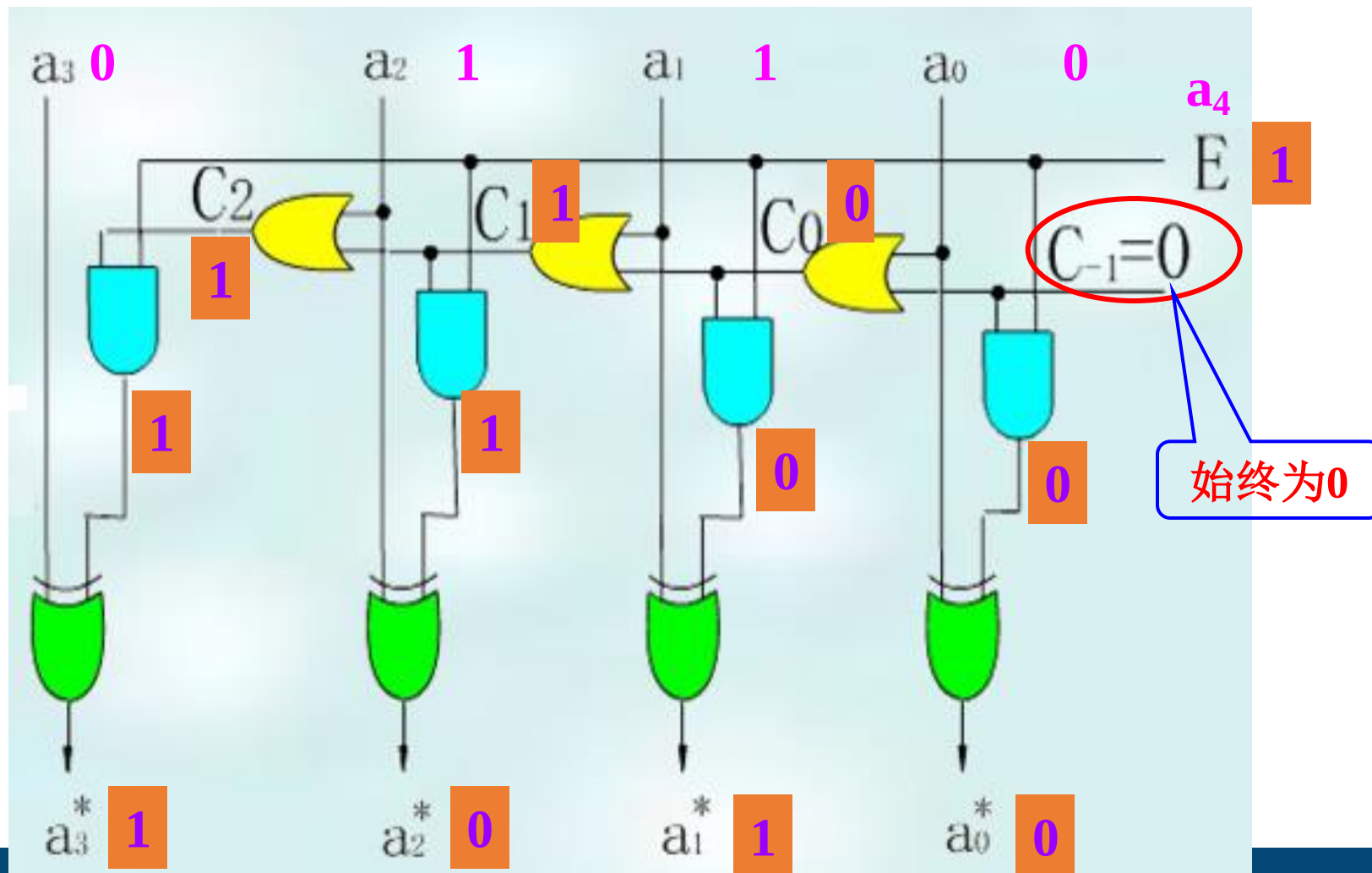
自右向左，第一个“1”的右侧所有数据位，均相同；
左侧所有数据位，均相反。

>>> 对2求补器电路逻辑

动画演示:

[2-3.swf](#)

➤ 采用按位扫描执行求补操作，E为控制信号线，由数据a的符号位控制；



>>> 2.3.2 直接补码并行乘法

直接使用补码的全部编码参与运算，不需要计算原值

➤ N位定点补码整数 $[N]_{\text{补}} = a_{n-1}a_{n-2}\dots a_1a_0$ ，其中 a_{n-1} 为符号位；

➤ 根据 $[N]_{\text{补}}$ 的符号，补码数 $[N]_{\text{补}}$ 和真值 N 的关系可以表示成：

$$N = \begin{cases} + \sum_{i=0}^{n-1} a_i 2^i & \text{当 } a_n = 0 ([M]_{\text{补}} \text{ 为正) 时} \\ - [1 + \sum_{i=0}^{n-1} (1 - a_i) 2^i] & \text{当 } a_n = 1 ([M]_{\text{补}} \text{ 为负) 时} \end{cases}$$
$$N = -a_n 2^n + \sum_{i=0}^{n-1} a_i 2^i$$

➤ 若把负权因数 -2^n 强加到符号位 a_n 上，那么就可以把上述方程组中的两个位置表达式合并成下面的统一形式：

>>> 一般化的全加器形式

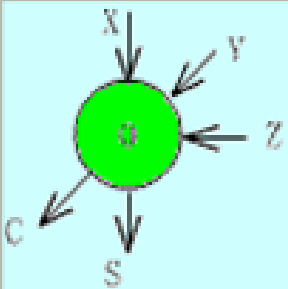
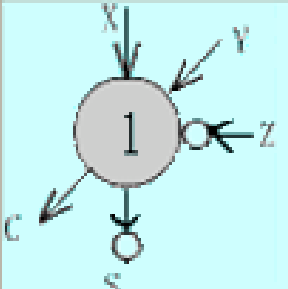
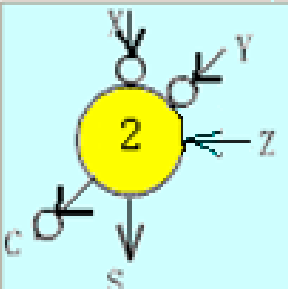
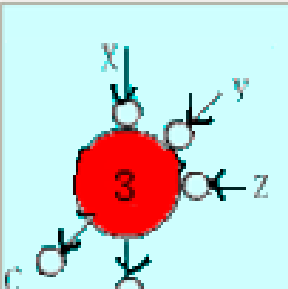
➤ 通过把正权或负权加到输入/输出端，可以归纳出四类加法单元。

$$1+1+1 = 10+1$$

$$1+1+(-1) = 10+(-1)$$

$$(-1)+(-1)+1 = (-10)+1$$

$$(-1)+(-1)+(-1) = (-10)+(-1)$$

类型	逻辑符号	操作
0类 加法器		$\begin{array}{r} X \\ Y \\ +) Z \\ \hline CS \end{array}$
1类 加法器		$\begin{array}{r} X \\ Y \\ +) -Z \\ \hline C(-S) \end{array}$
2类 加法器		$\begin{array}{r} -X \\ -Y \\ +) Z \\ \hline (-C)S \end{array}$
3类 加法器		$\begin{array}{r} -X \\ -Y \\ +) -Z \\ \hline (-C)(-S) \end{array}$

- 利用混合型的全加器构成的;
- 设两个5位二进制补码A、B, 可表示为:

$$A = (a_4)a_3a_2a_1a_0$$

$$\mathbf{B} = (b_4)a_3a_2a_1a_0$$

- 符号位 a_4 和 b_4 是带负权的, 加括号标注;

- **A × B的矩阵所示:**

[illegible]

带负权的项加括号表示



由于被加项(a_4b_0)是负的, 0可以看作-0, 所以用2类加法器。

由于此处有三个负的被加项, 所以用3类加法器

动画演示:

[2-5. swf](#)

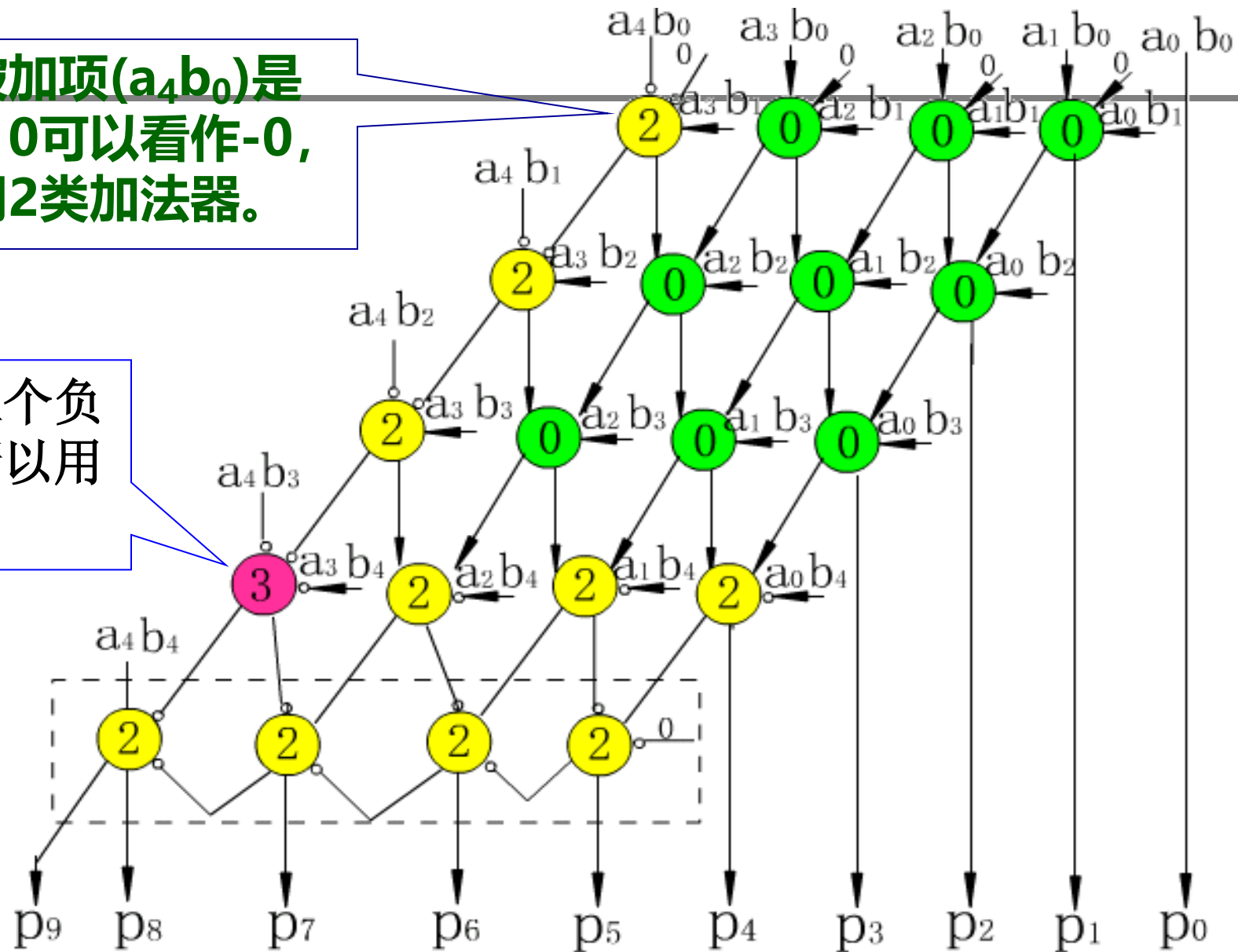


图2.8 5位乘5位的直接补码阵列乘法器逻辑

>>> 2020年考研统考题目 第43题 (13分)

43. 有实现 $x*y$ 的两个 C 语言函数如下:

```
unsigned umul ( unsigned x , unsigned y )  
{ return x*y; }
```

```
int imul ( int x, int y )  
{ return x * y; }
```

假定某计算机 M 中 ALU 只能进行加减运算和逻辑运算。请回答:

(1) 若 M 的指令系统中没有乘法指令, 但有加法、减法和位移等指令, 则在 M 上也能实现上述两个函数中的乘法运算, 为什么?

(2) 若 M 的指令系统中有乘法指令, 则基于 ALU、位移器、寄存器以及相应控制逻辑实现乘法指令时, 控制逻辑的作用是什么?

(3) 针对以下 3 种情况: (a) 没有乘法指令; (b) 有使用 ALU 和位移器实现的乘法指令; (c) 有使用阵列乘法器实现的乘法指令, 函数 umul() 在哪种情况下执行时间最长? 哪种情况下执行的时间最短? 说明理由

(4) n 位整数乘法指令可保存 $2n$ 位乘积, 当仅取低 n 位作为乘积时, 其结果可能会发生溢出。当 $n=32, x=2^{31}-1, y=2$ 时, 带符号整数乘法指令和无符号整数乘法指令得到的 $x*y$ 的 $2n$ 位乘积分别是什么 (用十六进制表示)? 此时函数 umul() 和 imul() 的返回结果是否溢出? 对于无符号整数乘法运算, 当仅取乘积的低 n 位作为乘法结果时, 如何用 $2n$ 位乘积进行溢出判断?

带符号整数指令乘法: $7FFF\ FFFFH * 2 = 0000\ 0000\ FFFF\ FFFE H$

无符号整数指令乘法: $7FFF\ FFFFH * 2 = 0000\ 0000\ FFFF\ FFFE H$

umul() 返回 $FFFF\ FFFE H$ 未溢出

imul() 返回 $FFFF\ FFFE H$ 有溢出(结果成了负数)

高 n 位全 0 则未产生溢出, 否则产生溢出

2.4 定点除法运算

2.4.1 原码除法算法原理 (串行除法)

2.4.2 并行除法器

>>> 2.4.1 原码除法算法原理

分析笔算除法

$$x = -0.1011 \quad y = 0.1101 \quad \text{求 } x \div y$$

$$\begin{array}{r} 0.1101 \overline{) 0.1101} \\ \underline{ 0.1101} \\ 0.0000 \\ \underline{ 0.0000} \\ 0.0000 \\ \underline{ 0.0000} \\ 0.0000 \\ \underline{ 0.0000} \\ 0.0000 \end{array}$$

✓ 商符单独处理

? 心算上商（比较大小）

? 余数不动低位补“0”
减右移一位的除数

? 上商位置不固定

$$x \div y = -0.1101 \quad \text{商符 心算求得}$$

$$\text{余数} = -0.00000111$$

笔算除法和机器除法的比较

笔算除法

商符单独处理

心算上商

余数 不动 低位补“0”
减右移一位 的除数

2 倍字长加法器

上商位置 不固定

机器除法

符号位异或形成

$|x| - |y| > 0$ 上商 1

$|x| - |y| < 0$ 上商 0

余数 左移一位 低位补“0”
减 除数

1 倍字长加法器

在寄存器 最末位上商

>>> 除法的数据约定

➤ 设n位被除数和除数用定点小数表示

被除数 $[x]_{\text{原}} = x_f \cdot x_{n-1} \cdots x_1 x_0$

除数 $[y]_{\text{原}} = y_f \cdot y_{n-1} \cdots y_1 y_0$

则商 $[Q]_{\text{原}} = (x_f \oplus y_f) + (0.x_{n-1} \cdots x_1 x_0) / (0.y_{n-1} \cdots y_1 y_0)$

式中, x_f 为被除数符号, y_f 为除数符号。

➤ 约定

□ 小数定点除法 $|x| < |y|$, 整数定点除法 $|x| > |y|$

✓ 避免商溢出

□ 被除数不等于 0, 除数不能为 0

常用串行除法的算法

1) 原码一位除法

恢复余数法

不恢复余数法 (加减交替法)

2) 补码一位除法

加减交替法

商校正法

控制端P选择运算类型

○ $P=0$ ，作加法运算

○P=1, 作减法运算

并行除法器的类型

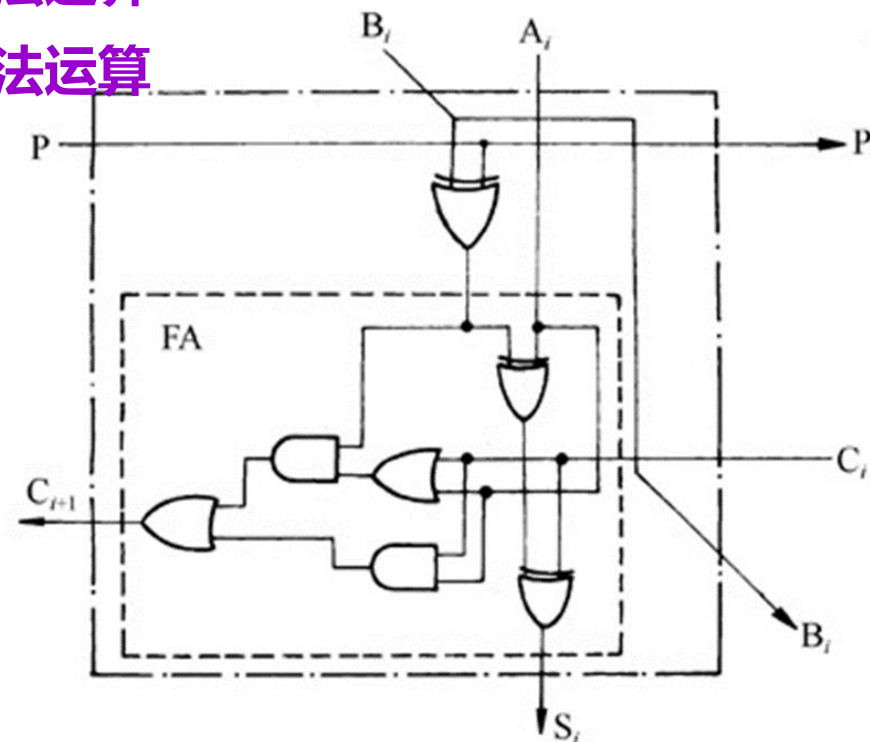
恢复余数阵列除法器

□不恢复余数阵列除法器

✓ 采用加减交替法原理

✓ 基本元件：可控加法/减法(CAS)单元

补码阵列除法器



(a) 可控加法/减法(CAS)单元的逻辑图

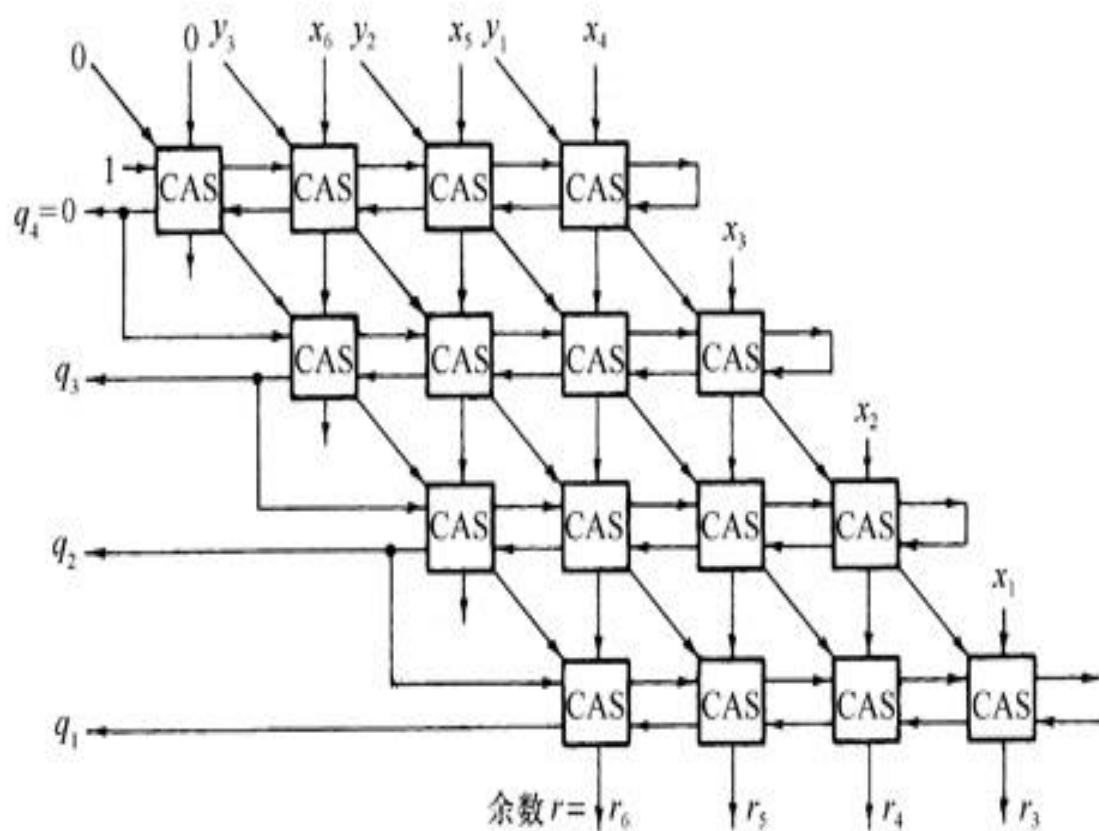
四个输入：

被加/减数 A_i 、加/减数 B_i 、低位进/借位 C_i

四个输出：

加/减数 B_j 、当位和/差 S_j 、向高位的进/借位 C_{j+1}

>>> 不恢复余数的阵列除法器



(b) 4 位除 4 位阵列除法器

- 被除数 $x = 0.x_6x_5x_4x_3x_2x_1$
 - 通过顶部一行和最右边对角线输入;
- 除数 $y = 0.y_3y_2y_1$
 - 沿对角线方向输入;
 - 逐行右移;
- 商 $q = 0.q_3q_2q_1$
 - 每次的加/减运算中产生;
- 余数 $r = 0.00r_6r_5r_4r_3$
 - 由最后一行产生;

2.5 定点运算器的组成

2.5.1 逻辑运算

2.5.2 多功能算术逻辑运算单元

2.5.3 内部总线

2.5.4 定点运算器的基本结构

>>> 串行加减法电路的问题

一、无法完成逻辑运算

二、进位延迟时间较长

>>> 2.5.1 逻辑运算

➤ 四种基本的逻辑运算：

□ 逻辑非、逻辑加（或）、逻辑乘（与）、逻辑异（异或）

➤ 算术运算与逻辑运算的差别：是否考虑进位

□ 算术运算： 每一位运算都需要考虑前一位的进位状态；

□ 逻辑运算： 每一位运算都是独立进行的，不考虑进位；

>>> 2.5.2 多功能算术逻辑运算单元 (74181)

➤一位全加器(FA)的逻辑表达式为:

$$F_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = A_i B_i + B_i C_i + C_i A_i$$

➤ALU设计思想:

□ A_i 和 B_i 先转换为组合函数 X_i 和 Y_i ;

✓ 该转换过程由参数 $S_0 S_1 S_2 S_3$ 控制;

□ 由 X_i , Y_i 和 C_i 进行全加器运算。

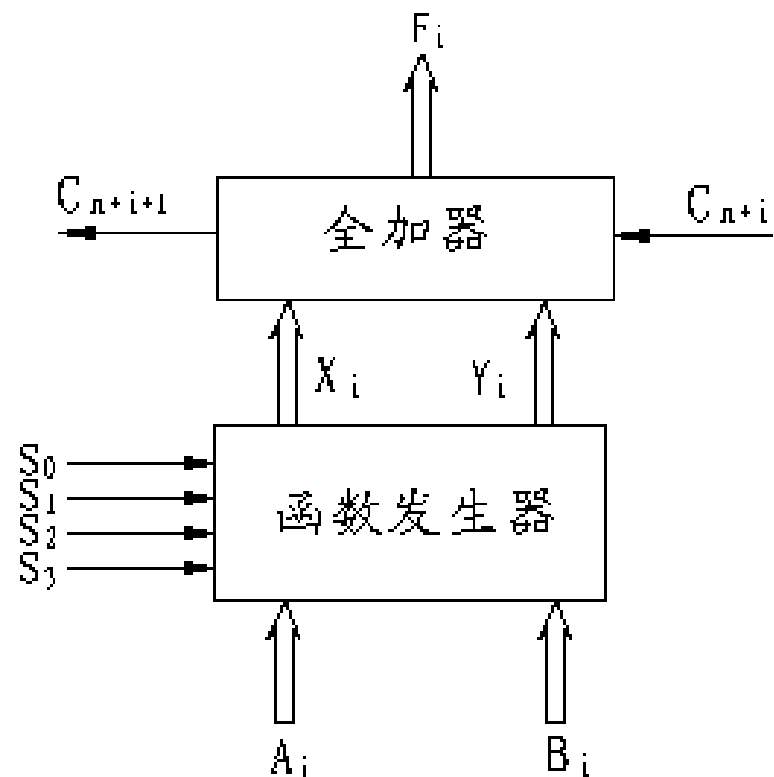
➤算术逻辑运算单元的逻辑式:

$$F_i = X_i \oplus Y_i \oplus C_i$$

$$C_{i+1} = X_i Y_i + Y_i C_i + X_i C_i$$

$$X_i = \overline{S_3 A_i B_i + S_2 A_i \overline{B_i}}$$

$$Y_i = \overline{A_i + S_0 B_i + S_1 \overline{B_i}}$$



S_1	S_0	Y_i	S_3	S_2	X_i
0	0	$\overline{A_i}$	0	0	1
0	1	$\overline{A_i} \overline{B_i}$	0	1	$\overline{A_i} + B_i$
1	0	$\overline{A_i} B_i$	1	0	$\overline{A_i} + \overline{B_i}$
1	1	0	1	1	$\overline{A_i}$

>>> ALU的运算讨论

➤ $S_1S_0=00, S_3S_2=00$ 若 $C_i=0$

$$\square Y_i = \neg A_i, X_i = 1$$

$$\square F_i = X_i \oplus Y_i \oplus C_i = 1 \oplus \neg A_i \oplus C_i = A_i$$

➤ $S_1S_0=11, S_3S_2=11$

$$\square Y_i = 0, X_i = \neg A_i$$

$$\square F_i = X_i \oplus Y_i \oplus C_i = (\neg A_i) \oplus 0 \oplus C_i = (\neg A_i) \oplus C_i$$

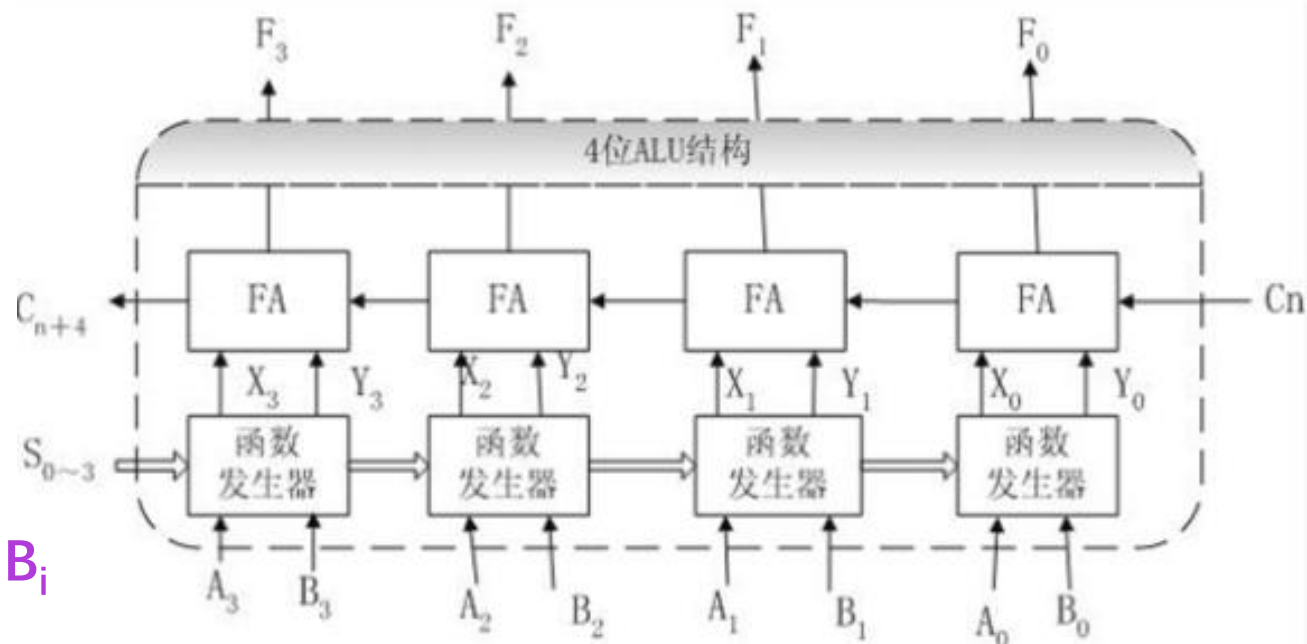
➤ $S_1S_0=01, S_3S_2=00$

$$\square Y_i = \neg A_i \neg B_i, X_i = 1$$

$$\square F_i = X_i \oplus Y_i \oplus C_i = 1 \oplus (\neg A_i \neg B_i) \oplus C_i = A_i + B_i$$

➤

S_1	S_0	Y_i	S_3	S_2	X_i
0	0	$\overline{A_i}$	0	0	1
0	1	$\overline{A_i} \overline{B_i}$	0	1	$\overline{A_i} + B_i$
1	0	$\overline{A_i} B_i$	1	0	$\overline{A_i} + \overline{B_i}$
1	1	0	1	1	$\overline{A_i}$



>>> 4位ALU 74181

➤控制端 详见课本P48 表2.4

□ $S_0 \sim S_3$ ：控制ALU的**运算方式**，如加1、A和B相加等；

□M：控制ALU的**运算类型**——算术运算、逻辑运算；

✓逻辑运算M=1，封锁 C_i ，送入 F_i 输入端的 $C_i=0$ ；

✓算术运算M=0， C_i 正常参与运算；

➤74181 ALU芯片有正逻辑、负逻辑之分

□ C_n 、P、G、 C_{n+i} 等信号的逻辑状态不同；

□具体的逻辑功能含义有差别；

动画演示：

[74181ALU.swf](#)

>>> 进位产生公式

➤由 $C_{i+1} = X_i Y_i + Y_i C_i + X_i C_i$ 推导:

$$\begin{aligned} C_{i+1} &= X_i Y_i + (X_i + Y_i) C_i \\ &= G_i + P_i C_i \end{aligned}$$

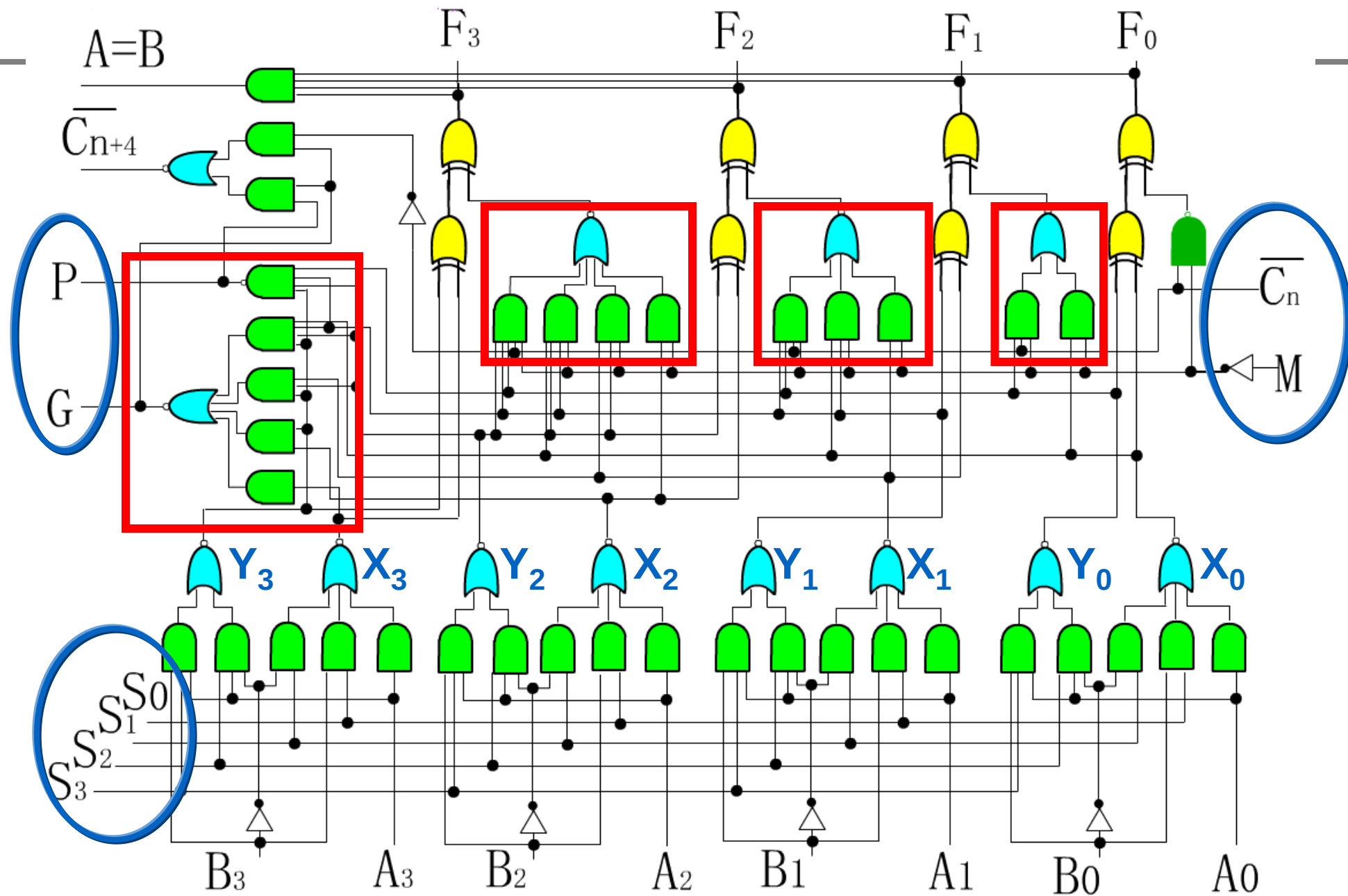
□ $G_i = X_i Y_i$ 为进位产生函数; $P_i = X_i + Y_i$ 为进位传递函数;

➤假设4位的ALU, 加数 $X_3 X_2 X_1 X_0$ 和 $Y_3 Y_2 Y_1 Y_0$ 、低位进位 C_0

$$\begin{aligned} C_4 &= G_3 + P_3 C_3 \\ &= G_3 + P_3 (G_2 + P_2 C_2) \\ &= G_3 + P_3 (G_2 + P_2 ((G_1 + P_1 C_1))) \\ &= G_3 + P_3 (G_2 + P_2 ((G_1 + P_1 (G_0 + P_0 C_0)))) \end{aligned}$$

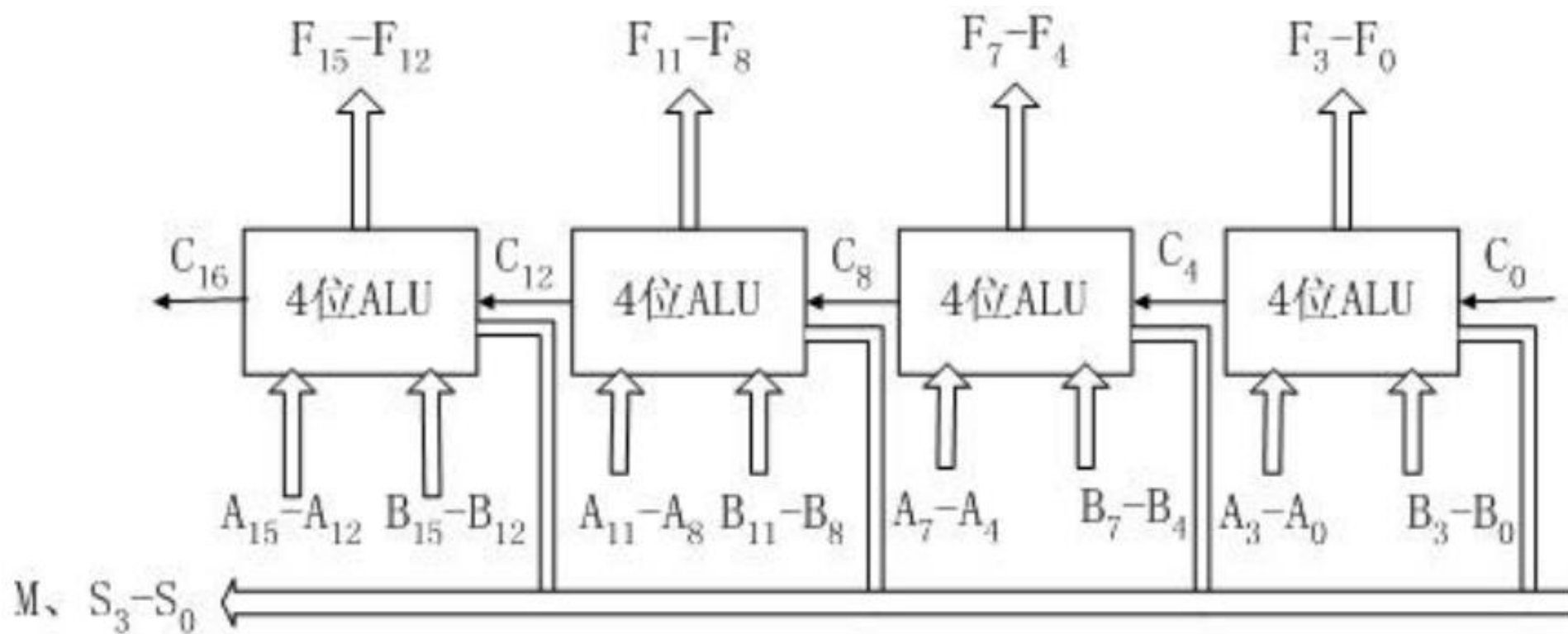
运算开始时, X_i 和 Y_i 均已知, 所有位的对应值都可以同步得到。

$$\begin{aligned} C_i &= f(G_j, P_j, C_0) \\ j &= 1, 2, \dots, i \end{aligned}$$



>>> 16位ALU结构

➤ 片内先行进位，片间串行进位



>>> 先行进位发生器

➤ 串行加法器(行波加法器):

- 将 n 个全加器级联, 得到 n 位加法器;
- 特点: 进位逐位传递, 速度慢。

➤ 超前进位加法器: 74LS182

□ 结构形式

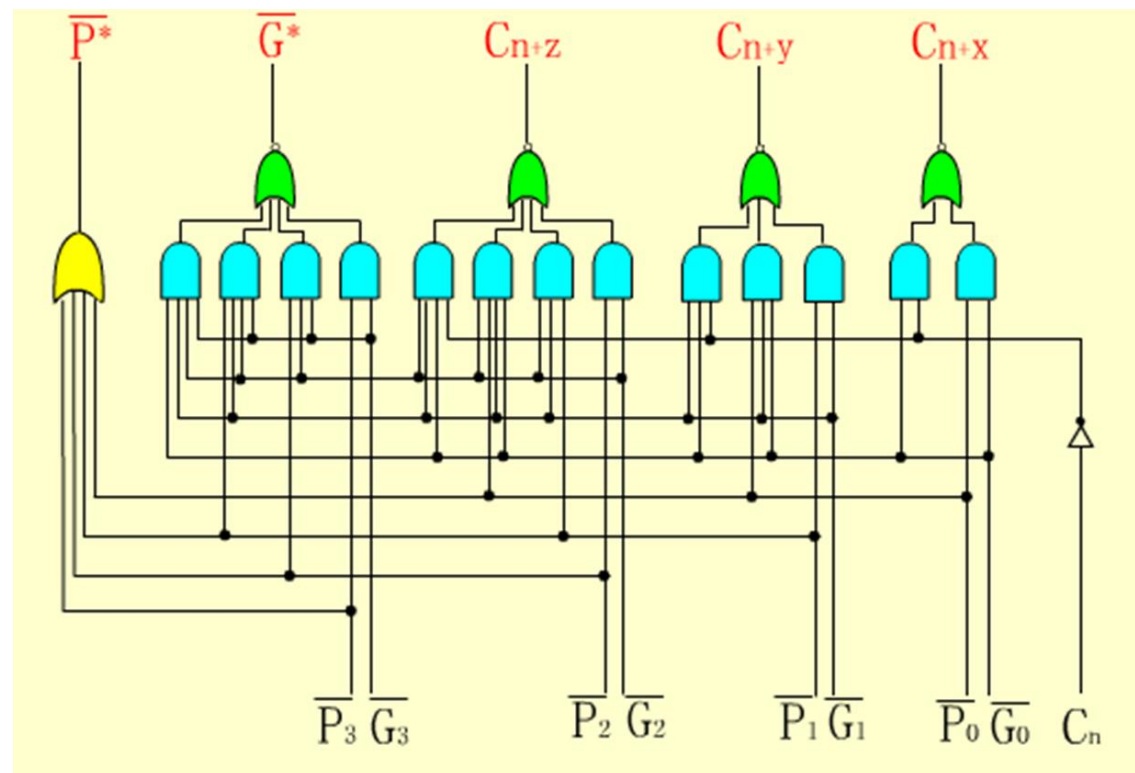
- ✓ 多片先行进位加法器级联而成;
- ✓ 片内先行进位, 片间先行进位部件成组进位;

□ 特点

- ✓ 和的所有位可同时计算, 运行速度快。

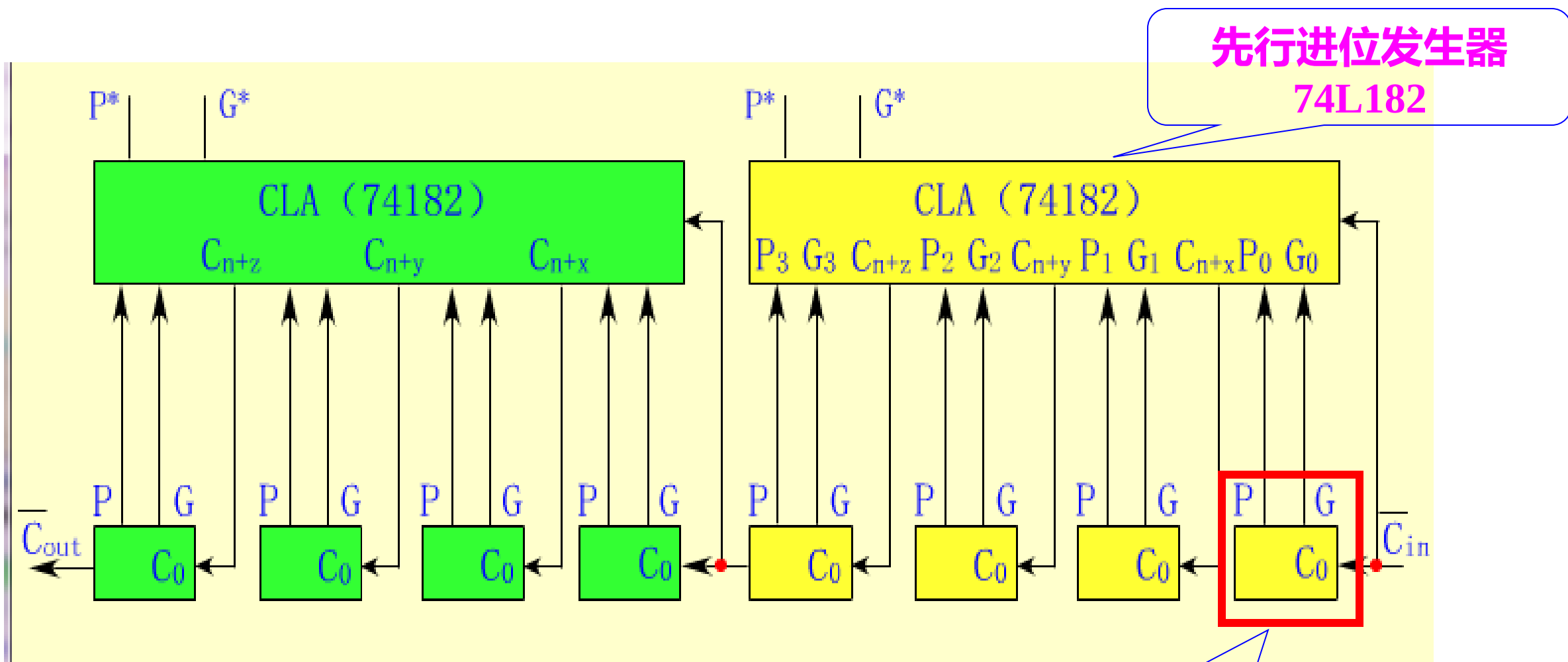
74L182

成组先行进位部件CLA



课本 P47

>>> 2个16位全先行进位逻辑级联组成的32位ALU



>>> 2.5.3 内部总线

➤ 根据总线所在位置分

□ 内部总线：CPU内各部件的连线

□ 外部总线：系统总线

✓ CPU与存储器、I/O系统之间的连线

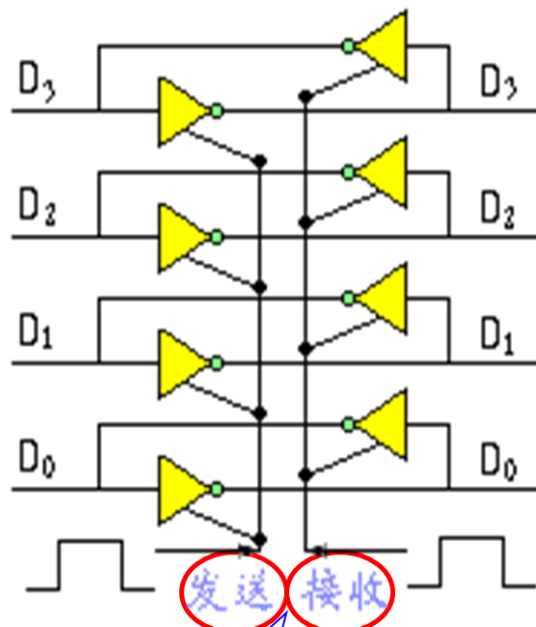
➤ 按总线的逻辑结构分

□ 单向总线：信息只能向一个方向传送

□ 双向总线：信息可以分两个方向传送，
既可以发送数据，也可以接收数据

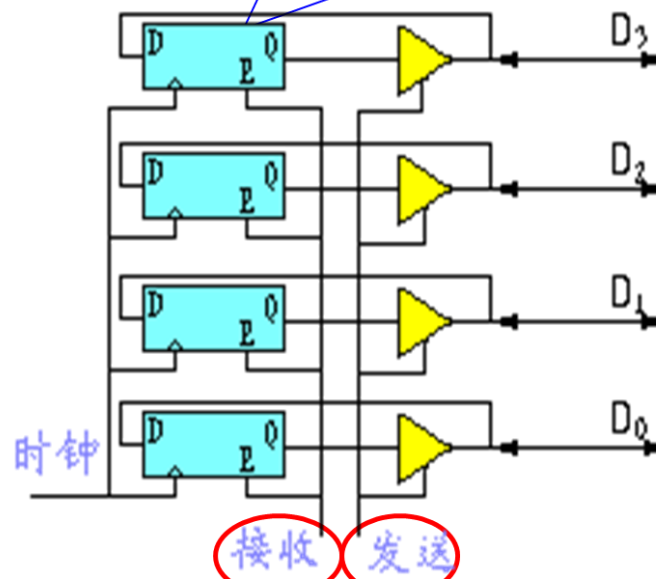
DE触发器：

控制端E用作控制D端的输入能否有效



(a) 带有缓冲器的双向数据总线

任何时候二者
不能同时有效



(b) 带有锁存器的4位双向数据总线

任何时候二者
不能同时有效

>>> 2.5.4 定点运算器的基本结构

课本P50 图2.14
运算器的三种结构形式

➤ 单总线结构的运算器

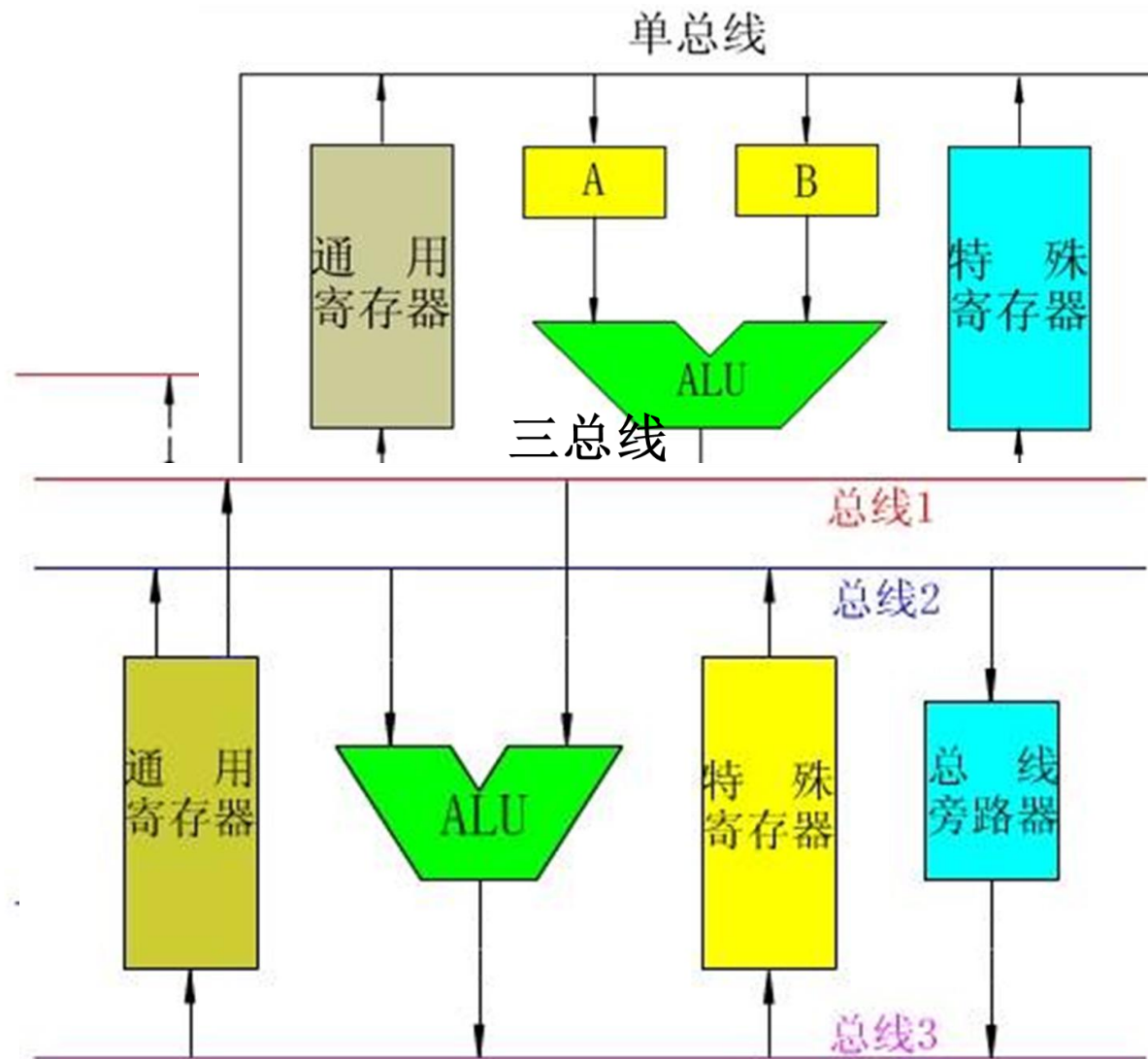
- 同一时间内，总线上只能有一个操作数。
- 缺点：操作速度较慢。
- 优点：控制电路比较简单。

➤ 双总线结构的运算器

- 两个操作数可以同时送到ALU进行运算。

➤ 三总线结构的运算器

- ALU的两个输入分别由两条总线提供；
- ALU的输出与第三条总线相连。



2.6 浮点运算方法和浮点运算器

2.6.1 浮点加法、减法运算

2.6.2 浮点乘法、除法运算

2.6.3 浮点运算流水线

2.6.4 浮点运算器实例

某浮点机，采用规格化浮点数表示，阶码用移码表示，尾数用原码表示。下列哪个数的表示不是规格化浮点数？（ ）

- ☐ A 11111110, 11000...00
- ☒ B 00111111, 10111...01
- ☐ C 10000001, 01111...01
- ☐ D 01111111, 010000...10

提交

2020年考研统考题目 第13题

13. 已知带符号整数用补码表示，float型数据用IEEE754标准表示，假定变量x的类型只能是int或float。当x的机器数为C800 0000H时，x的值可能是（ ）

☒ A -7×2^{27}

☐ B -2^{16}

☐ C 2^{17}

☐ D 25×2^{27}

提交

>> 2020年考研统考题目 第13题

13. 已知带符号整数用补码表示，float型数据用IEEE754标准表示，假定变量x的类型只能是int或float。当x的机器数为C800 0000H时，x的值可能是（ ）

E														
D ₃₁	D ₃₀	D ₂₉	D ₂₈	D ₂₇	D ₂₆	D ₂₅	D ₂₄	D ₂₃	~D ₂₀	D ₁₉ ~D ₁₆	D ₁₅ ~D ₁₂	D ₁₁ ~D ₈	D ₇ ~D ₄	D ₃ ~D ₀
1	1	0	0	1	0	0	0	0	0	0	0	0	0	0
-	0	1	1	1	0	0	0							

➤ 若为float数据，则s=1，E=1001 0000，M=1.0

$$\square x = -1 \times 2^{17}$$

➤ 若为int数据，按补码规则其真值为-01110...0

$$\square x = -(2^{29} + 2^{28} + 2^{27}) = -7 \times 2^{27}$$

>>> 2.6.1 浮点数加减运算

➤ 设有两个浮点数 $X = M_x \times 2^{E_x}$ 和 $Y = M_y \times 2^{E_y}$ ，且 $E_x > E_y$

➤ 若要求 $X \pm Y$ 的结果 S ，则

$$\square S = X \pm Y = M_s \times 2^{E_s} \quad \text{其中, } E_s = E_x, M_s = M_x \pm (M_y \text{ SHR } (E_x - E_y))$$

➤ 浮点数加减运算的步骤

□ 零操作数检查

一个操作数为0，则不必运算，节省运算时间

□ 两操作数对阶

小数点位置对齐，即阶码相同，为加减运算做准备

□ 尾数相加减

定点小数相加减，注意尾数的溢出判断

□ 结果的规格化

将非规格化的结果值进行规格化处理

□ 结果的舍入处理

尽量保持计算后数据的精确度

□ 结果的溢出判断

以阶码溢出为标准，尾数溢出则继续规格化处理

>>> 浮点数加减运算——两操作数对阶

➤ 对阶的原则

□ 以较大的阶码为标准，调整阶码较小的数据；

✓ 避免阶码较大的浮点数的尾数左移，导致最高有效数位丢失；

增大阶码，尾数右移

➤ 具体操作

□ 求阶差 $\Delta E = E_X - E_Y$

□ 调整阶码较小的数据

✓ 若 $\Delta E > 0$ ，则 M_Y 右移 ΔE 位，结果的阶码为 E_X

✓ 若 $\Delta E < 0$ ，则 M_X 右移 $|\Delta E|$ 位，结果的阶码为 E_Y

➤ 例，X—— $E_X = 0\ 001$ ， $M_X = 0.101$ ；Y—— $E_Y = 0\ 011$ ，尾数 $M_Y = 0.111$

□ 阶差 $\Delta E = E_X - E_Y = 0\ 001 - 0\ 011 = -10$

□ X尾数 M_X 右移2位， $M_X = 0.001\ 01$

< 0

>>> 浮点数加减运算——结果的规格化处理

➤ 两尾数加减的结果有两种情况

□ 尾数溢出 $\longrightarrow$ 两符号位为01或10 $\longrightarrow$ 右规

✓ 尾数右移1位，阶码加1

□ 尾数为非规格化数据 $\longrightarrow$ 补码表示的符号位与最高数值位相同 $\longrightarrow$ 左规

✓ 尾数左移1位，阶码减1，直至数值位最高位与符号位相反。

➤ 接上例，对阶后 $E=E_Y=0\ 011$ ，尾数 $M_X=0.001\ (01)$ ， $M_Y=0.111$

□ 尾数求和： $M_S = M_X + M_Y = 00.001\ 01 + 00.111 = 01.000\ 01$

□ 两符号位相反，应进行右规1位的操作

□ 则 $M_S = 00.100\ (001)$ ， $E_S = 011+1 = 0\ 100$

>>> 浮点数加减运算——结果的舍入处理

➤ 在对阶或右规操作时，会使加数或结果的尾数低若干位移出，影响精度，常用两种舍入处理方法

□ 方法1：0舍1入法

- ✓ 保留右移时的移出位，若最高位为1，则尾数加1；否则舍去；
- ✓ 特点：精度较高，但需要记录所有的移出位。

□ 方法2：恒置1法

- ✓ 若之前步骤有右移操作，则直接将结果的最低位置1；
- ✓ 特点：精度较0舍1入法较低，但应用简单。

➤ 接上例，结果的尾数 $M_S = 00.100\ 001$

□ 0舍1入法： $M_S = 00.100$

□ 恒置1法： $M_S = 00.101$

>>> 浮点数加减运算——结果的溢出判断

➤ 尾数溢出

□ 在规格化处理时，通过完成右规完成；

➤ 阶码溢出

□ 上溢(结果绝对值太大)——置上溢标志，结束；

□ 下溢(结果绝对值太小)——置机器零；

□ 正常——运算结束；

➤ 接上例，运算结果的阶码 $E_S = 011 + 1 = 0100$

□ 未溢出！



>>> 课本P53[例2.28]
设 $x = 2^{010} \times 0.11011011$, $y = 2^{100} \times (-0.10101100)$, 求 $x + y$ 。

➤ 设浮点数的阶码用双符号位，尾数用单符号位的补码表示；

□ $[X]_{\text{浮}} = 00\ 010, 0.11011011$

□ $[Y]_{\text{浮}} = 00\ 100, 1.01010100$

① 求阶差并对阶

○ $\Delta E = E_X - E_Y = [E_X]_{\text{补}} + [-E_Y]_{\text{补}}$
 $= 00\ 010 + 11\ 100 = 11\ 110$

$\Delta E = -2 < 0$
则 E_Y 为结果阶码，修改浮点数 X

○ 则浮点数 X ，应使 M_X 右移2位， E_X 加2，
 $[X]_{\text{浮}} = 00\ 100, 0.00110110(11)$, $E_S = 00\ 100$

② 尾数求和：

□ $M_S = 1.10001010(11)$

$$\begin{array}{r} 00.00110110(11) \\ + 11.01010100 \\ \hline 11.10001010(11) \end{array}$$

>>> 课本P53[例2.28]
设 $x = 2^{010} \times 0.11011011$, $y = 2^{100} \times (-0.10101100)$, 求 $x + y$ 。

③ 和的规格化处理

$$M_S = 11.10001010(11)$$

□ 结果尾数的符号位与最高数值位相同, 应左规1位

□ 则 $M_S = 1.00010101(10)$, $E_S = 00\ 100 - 1 = 00\ 011$ 。

④ 舍入处理

□ 若采用0舍1入法

$$M_S = 1.00010110$$

$$\begin{array}{r} 1.00010101 \\ + \qquad \qquad \qquad 1 \\ \hline 1.00010110 \end{array}$$

⑤ 结果溢出判断

□ 阶码符号位为00, 和不溢出

$$\begin{aligned} \text{最终结果 } S = X + Y &= 00\ 011, 1.00010110 \\ &= 2^{011} \times (-0.11101010) \end{aligned}$$

>>> 课堂练习

已知: $x = -0.1000101 \times 2^{-111}$

$y = +0.0001010 \times 2^{-100}$

①用补码运算求 $x + y = ?$ 并判断是否溢出?

②用补码运算求 $x - y = ?$ 并判断是否溢出?

【解】

假设两数均以补码表示, 且采用双符号位, 先将它们变为规格化浮点数, 则它们的浮点表示分别为

$[x]_{\text{浮}} = 11\ 001, \ 11.0111011$

$[y]_{\text{浮}} = 11\ 100, \ 00.0001010 = 11\ 001, \ 00.1010000$

>>> 已知 $[x]_{\text{浮}} = 11\ 001, 11.0111011$, $[y]_{\text{浮}} = 11\ 001, 00.1010000$
求 $x + y$ 。

<1> 求阶差并对阶

$E_x = E_y$, 即 ΔE 为0, 阶码相等, 无需对阶。

<2> 尾数求和

$M_s = 00.0\ 0\ 0\ 1\ 0\ 1\ 1$

$$\begin{array}{r} 11.0\ 1\ 1\ 1\ 0\ 1\ 1 \\ +\quad 00.1\ 0\ 1\ 0\ 0\ 0\ 0 \\ \hline 00.0\ 0\ 0\ 1\ 0\ 1\ 1 \end{array}$$

<3>规格化处理

尾数运算结果的符号位与最高数值位同值, 应执行左规处理;

$M_s = 00.1011000$, $E_s = 10\ 110$

<4>舍入处理: 没有丢失位, 不必进行舍入。

<5>判溢出: 阶码符号位为10, 发生下溢, 故需置机器0。

>>> 已知 $[x]_{\text{浮}} = 11\ 001, 11.0111011$, $[y]_{\text{浮}} = 11\ 001, 00.1010000$
求 $x - y$ 。

<1> 求阶差并对阶: $E_x = E_y$, 即 ΔE 为 0, 阶码相等, 无需对阶。

<2> 尾数求差

$$[-M_y]_{\text{补}} = 11.0110000, \quad M_s = 10.1101011 \quad + \quad \begin{array}{r} 11.0111011 \\ 11.0110000 \\ \hline 10.1101011 \end{array}$$

<3> 规格化处理: 两位符号位不同, 应执行右规1位的处理。

$$M_s = 11.0110101 \ (1), \quad E_s = 11\ 010.$$

<4> 舍入处理

0舍1入法
丢失位高位为1, 故应进位, $M_s = 11.0110110$ 。

<5> 判溢出

阶码符号位为 11, 无溢出, 故结果为:

$$x - y = -0.1001010 \times 2^{-6}.$$

2009年考研统考题目 第13题

设浮点数的阶码和尾数均采用补码表示，且位数分别为5位和7位（均含2位符号位）。若有两个数 $X=2^7 \times 29/32$ ， $Y=2^5 \times 5/8$ ，则有浮点加法计算 $X+Y$ 最终结果是（ ）。

- ☐ A 00111 1100010
- ☐ B 00111 0100010
- ☐ C 01000 0010001
- ☒ D 发生溢出

$$X=1\ 1101 \times 2^{+2}$$

$$Y=101 \times 2^{+2}$$

提交

2015年考研题 第14题

下列有关浮点数加减运算的叙述中，正确的是___。

- I. 对阶操作不会引起阶码上溢或下溢
- II. 右规和尾数舍入都可能引起阶码上溢
- III. 左规时可能引起阶码下溢
- IV. 尾数溢出时结果不一定溢出

A 仅 II、III

C 仅 I、III、IV

B 仅 I、II、IV

D I、II、III、IV

提交

>>> 2.6.2 浮点乘法、除法运算

➤ 设两个浮点数： $X = M_x \times 2^{E_x}$ 和 $Y = M_y \times 2^{E_y}$

□ 乘法： $X \times Y = 2^{E_x + E_y} \times (M_x \cdot M_y)$

□ 除法： $X \div Y = 2^{E_x - E_y} \times (M_x \div M_y)$

➤ 浮点数乘除运算的步骤

□ 0 操作数检查；可直接得到0结果

□ 阶码加/减操作；阶码为补码或移码形式，可直接加减运算

□ 尾数乘/除操作；尾数为原码或补码形式，采用定点运算器

□ 结果规格化、舍入和溢出处理；

➤ 本小节其他内容自学！

>>> 2.6.3 浮点运算流水线

➤ 流水线原理

□ 把输入的任务分割成一系列子任务，使各个子任务能在流水线的各个阶段并发地执行。

□ 实际上是在计算机上实现时间并行性的一种非常经济的方法。

➤ 流水线相关内容在控制器章节中讲解。

>>> 本章小结

- 掌握数据的不同表示方法，并能在不同的表示形式间转换；
 - 真值和机器数；原补移码、BCD码、定点数等表示
 - 浮点数的表示、取值范围、规格化、IEEE754标准
- 理解数据校验的原理和方法
 - 奇偶校验、海明校验
- 掌握定点数的运算，了解相关运算器的构成
 - 补码加减法运算、溢出的检测和判断方法、加减法电路
 - 了解定点数的串行、并行乘除法运算方法，及运算器构成
- 掌握浮点数的加减法运算方法和步骤